



ESCUELA DE INGENIERÍA DE FUENLABRADA

GRADO EN INGENIERÍA DE ROBÓTICA SOFTWARE

TRABAJO FIN DE GRADO

**EVALUACIÓN DE LA ROBUSTEZ Y EFECTIVIDAD DE MODELOS
FUNDACIONALES PARA LA DETECCIÓN DE PÓLIPOS USANDO
IMÁGENES DE COLONOSCOPIAS**

Autor: Santiago Prieto Núñez

Tutor: Luis Bote Curiel

Co-tutor: Cristian David Chushig Muzo

Curso académico 2024/2025

Health is the core of human development.

– Gro Harlem Brundtland

Agradecimientos

Querría empezar recordando a todos aquellos profesores que a lo largo de mi vida académica han demostrado su vocación y esfuerzo en cada una de las distintas aulas que he pisado. Su trabajo me ha inspirado y guiado hasta este momento, gracias también a todos aquellos compañeros y amigos que han hecho este camino más sencillo, su cariño y compañía bien merece estas líneas.

A mis familiares que gracias a su cariño y esfuerzo me ha dado el privilegio de poder terminar esta ingeniería. A Kike por el esfuerzo que ha hecho y hace por esta familia, su presencia ha sido una guía desde que está en mi vida.

Gracias a mis dos hermanos, compañeros de vida cada uno en distintos ámbitos y a los cuales debo mi amor y respeto, A ti Inés por todos los años de compañía y ayuda. A ti Fernando, doy gracias por haber crecido juntos, te admiro y te quiero hermano.

A mamá, por el infinito sacrificio y esfuerzo que has hecho por nosotros, puedo decir que has sido y seguirás siendo una madre ejemplar. Y aunque puede que no sepas exactamente que he hecho. Puedo decirte hoy con orgullo: Mamá ya soy ingeniero.

Resumen

El cáncer colorrectal representa aproximadamente el 10 % de los casos de cáncer a nivel mundial y es la segunda causa de muerte por cáncer cada año. No obstante, puede prevenirse en gran medida mediante un diagnóstico temprano y un tratamiento adecuado. En este contexto, el uso de herramientas de Inteligencia Artificial (IA) para la detección precoz de pólipos durante las colonoscopias se ha consolidado como una estrategia eficaz, mejorando la identificación de lesiones y apoyando la toma de decisiones clínicas.

Este Trabajo de Fin de Grado analiza la robustez y efectividad de diversos modelos fundacionales basados en *Vision Transformers* (ViT) para la detección automática de pólipos en imágenes endoscópicas, comparando su efectividad frente a *Redes Neuronales Convolucionales* (CNN), comúnmente usadas en este contexto. Se emplean imágenes obtenidas tanto con luz blanca (*White Light*, WL) como con imagen de banda estrecha (*Narrow Band Imaging*, NBI), los dos tipos de luz más utilizados durante el diagnóstico. El estudio se utiliza un conjunto de datos que principalmente contiene el *datasets* bajo solicitud *Piccolo*, este contiene segmentaciones manuales y metadatos clínicos, con un total de 3433 imágenes. Para aumentar la diversidad y robustez del entrenamiento, se incorporan además 579 imágenes del conjunto de datos *CVC* y 2814 imágenes de *PolypDB*, lo que permite obtener una muestra más completa de los distintos tipos de lesiones observadas durante la colonoscopia.

Se comparan los resultados obtenidos por diferentes modelos fundacionales más antiguos, como *YolosBBoxOnly*, *ViT-Small*, *ViT-Large*, y modelos más modernos como *PVT* y *DeiT*, frente a arquitecturas CNN como *YOLOv8* y *DenseNet121*, utilizando distintos conjuntos de datos. Esto permite realizar un análisis exhaustivo del rendimiento de estas nuevas arquitecturas en el contexto de la detección de pólipos durante colonoscopias, así como su comparación con enfoques más convencionales.

Los resultados evidencian que los modelos fundacionales más antiguos, como *YolosBBoxOnly*, *ViT-Large* y *ViT-Small*, superan el rendimiento de las CNN en un 9.96 %. En cambio, modelos *transformer* más recientes como *DeiT* y *PVT*, presentan mejoras significativas: un **14.88 %** y un **16.04 % por encima del rendimiento de las CNN**, alcanzando **resultados superiores al 81 %**. Estos resultados confirman que las arquitecturas *transformer* más modernas ofrecen un rendimiento muy prometedor, con potencial para superar a las herramientas de detección actuales.

Abstract

Colorectal cancer accounts for approximately 10% of cancer cases worldwide and is the second leading cause of cancer-related death each year. However, it can largely be prevented through early diagnosis and appropriate treatment. In this context, the use of Artificial Intelligence (AI) tools for the early detection of polyps during colonoscopies has emerged as an effective strategy, improving lesion identification and supporting clinical decision-making.

This Bachelor's Thesis analyzes the robustness and effectiveness of various foundation models based on Vision Transformers (ViT) for the automatic detection of polyps in endoscopic images, comparing their performance with that of Convolutional Neural Networks (CNN), which are commonly used in this field. The study uses images obtained with both white light (WL) and narrow band imaging (NBI), the two most commonly used lighting types during diagnosis. The primary dataset used in this study is the Piccolo dataset (available upon request), which includes manual segmentations and clinical metadata, with 3,433 images in total. To increase training diversity and robustness, an additional 579 images from the CVC dataset and 2,814 images from PolypDB are included, allowing for a more comprehensive sampling of the various lesion types observed during colonoscopy.

The results of older foundation models, such as YolosBBoxOnly, ViT-Small, and ViT-Large, are compared with more modern models like PVT and DeiT, as well as with CNN architectures such as YOLOv8 and DenseNet121, using different datasets. This enables a thorough performance analysis of these new architectures in the context of polyp detection during colonoscopies, as well as their comparison with more conventional approaches.

The results show that older foundation models such as YolosBBoxOnly, ViT-Large, and ViT-Small outperform CNNs by 9.96%. In contrast, more recent transformer models such as DeiT and PVT show significant improvements: **14.88 % and 16.04 % above the performance of CNNs**, respectively, reaching **performance levels above 81 %**. These results confirm that modern transformer architectures offer very promising performance, with the potential to surpass current detection tools.

Índice general

Agradecimientos

Resumen

Índice de figuras v

Índice de cuadros ix

Lista de acrónimos y abreviaturas xii

1. Introducción y objetivos 1

1.1. Contexto y motivación 1

1.2. Objetivos 3

1.3. Metodología 3

1.4. Estructura de la memoria 5

2. Conceptos previos 7

2.1. Cáncer colorrectal 7

2.1.1. Origen y evolución 8

2.1.2. Fases de desarrollo del pólipo durante el CCR 10

2.1.3. Métodos de detección 11

2.2. Aprendizaje automático 13

2.2.1.	Definiciones generales	13
2.2.2.	Tipos de aprendizaje automático	16
2.2.3.	Técnicas del aprendizaje automático	17
2.2.4.	Conjunto de <i>train</i> , <i>validation</i> y <i>test</i>	18
2.2.5.	Métricas y Evaluación de los modelos	22
2.2.6.	Arquitecturas y tecnologías específicas	27
2.3.	Proceso de entrenamiento	34
2.3.1.	<i>Forward-propagation</i>	35
2.3.2.	Cálculo de pérdida	36
2.3.3.	Actualización de pesos	37
2.4.	El bucle de entrenamiento	38
3.	Métodos	41
3.1.	Software y entorno de desarrollo	42
3.2.	Implementación de modelos	47
3.2.1.	YolosBBoxOnly	47
3.2.2.	YOLOv8	49
3.2.3.	ViT-Large y ViT-Small	50
3.2.4.	DenseNet121	52
3.2.5.	Data-efficient Image Transformer (DEiT)	54
3.2.6.	Pyramid Vision Transformer (PVT)	55
3.3.	Síntesis de los modelos presentados	57
4.	Base de datos y análisis descriptivo	59
4.1.	Piccolo	59
4.2.	PolypDB	63
4.3.	CVC	65
4.4.	Composición de los <i>splits</i>	66

5. Experimentos y resultados	71
5.1. <i>Benchmark</i>	71
5.1.1. <i>Benchmark</i> Piccolo	72
5.1.2. <i>Benchmark</i> Piccolo+	72
5.2. Resultados y análisis	73
5.2.1. Resultados del <i>benchmark</i> Piccolo	73
5.2.2. Resultados del <i>benchmark</i> Piccolo+CVC+PolypDB	75
6. Conclusiones y líneas futuras	79
6.1. Conclusiones	79
6.2. Líneas futuras	80
Bibliografía	83

Índice de figuras

1.1. Incidencia estimada de cánceres en España para los años 2020 y 2040, por sexos.	2
1.2. Diagrama de Gantt que muestra las tareas realizadas durante el desarrollo del TFG.	5
2.1. Partes del colon e incidencia de tipo de cáncer.	8
2.2. Tipos de pólipos bajo microscopio y con tinción con hematoxilina y eosina. . .	9
2.3. Morfología de la clasificación de París.	9
2.4. Capas del colon.	10
2.5. Evolución del CCR.	11
2.6. Imágenes de COMBO-3 y prueba inmunoquímica fecal.	12
2.7. Arquitectura simplificada de una MLP.	15
2.8. <i>Splits</i> de un <i>dataset</i> bien estructurado.	20
2.9. Ejemplos de formatos de cajas delimitadoras.	20
2.10. Máscara correspondiente a la muestra de la Fig.2.9b del <i>dataset</i> Piccolo [1]. . .	21
2.11. Ejemplo de una buena y mala IoU en el conjunto de <i>train</i> para el modelo ViT- <i>Large</i>	23
2.12. Eficiencia de las distintas métricas de la familia IoU.	25
2.13. Esquema simplificado del descenso por gradiente.	26
2.14. Ejemplo de curvas de aprendizaje que presentan <i>overfitting</i>	27
2.15. Ejemplo de <i>embeddings</i> para lenguaje natural [2].	28
2.16. Proceso de <i>self-attention</i> con propósito ilustrativo.	29
2.17. Diagrama de <i>Multi-head attention</i>	30

2.18. Esquema del <i>encoder</i> de una arquitectura <i>Transformer</i>	31
2.19. Esquema de la arquitectura original <i>encoder-decoder</i>	32
2.20. Esquema una arquitectura CNN aplicada a la neuroimagen.	33
2.21. Esquema de las diferentes arquitecturas ViT.	34
2.22. Esquema de una red MLP en el paso <i>forward-propagation</i>	36
2.23. Esquema de una red MLP en el paso <i>back-propagation</i>	38
3.1. Inicializar la clase IDP.	43
3.2. Cargamos las imágenes del conjunto de <i>train</i> del <i>dataset</i> de Piccolo en la clase IDP.	44
3.3. Análisis de la composición del <i>dataset</i> de Piccolo usando IDP.	44
3.4. División del <i>dataset</i> en Dataloaders usando la clase IDP.	45
3.5. Definición de la <i>loss function</i> y el optimizador.	46
3.6. Uso de la clase <i>TrainModel</i> para entrenar un modelo.	46
3.7. Arquitectura del modelo ViT.	48
3.8. Esquema de la arquitectura usada en <i>YOLOv8</i>	50
3.9. Esquema del <i>encoder</i> de una arquitectura <i>Transformer</i>	51
3.10. Esquema de la composición de la cabeza de detección.	52
3.11. Esquema de la arquitectura y conexión entre capas de <i>DenseNet</i>	53
3.12. Esquema de la arquitectura DeiT.	55
3.13. Cabeza de atención estándar (izquierda), SRA (derecha).	56
3.14. Esquema de la arquitectura PVT.	57
4.1. Comparación de rendimiento del modelo <i>ViT-Large</i> antes y después del prepro- cesado.	61
4.2. Gráficos de la composición del <i>dataset</i> <i>Piccolo</i>	61
4.3. Distribución de imágenes por tipo de lesión en el <i>dataset</i> de <i>Piccolo</i>	63
4.4. Gráficos de la composición del <i>dataset</i> <i>PolypDB</i>	64
4.5. Gráficos de la composición del <i>dataset</i> <i>CVC</i>	66

4.6. Porcentaje de imágenes de cada tipo de lesión en cada <i>split</i>	67
4.7. Ejemplos de formatos de cajas delimitadoras.	68
5.1. Resultados del rendimiento de los modelos en <i>train</i> y <i>validation</i> en el <i>benchmark</i> Piccolo.	74
5.2. Resultados del rendimiento de los modelos en <i>train</i> y <i>validation</i> en el <i>benchmark</i> Piccolo+CVC+PolypDB.	76

Índice de cuadros

4.1.	Resumen de la composición del <i>dataset</i> <i>Piccolo</i>	62
4.2.	Resumen datos clínicos de <i>Piccolo</i>	63
4.3.	Resumen de la composición de <i>PolypDB</i>	65
4.4.	Resumen de la composición de <i>CVC</i>	66
4.5.	Comparación de <i>splits</i> del <i>dataset</i>	69
5.1.	Rendimiento de las arquitecturas en el <i>benchmark</i> <i>Piccolo</i>	74
5.2.	Rendimiento de las arquitecturas en el <i>benchmark</i> <i>Piccolo+CVC+PolypDB</i> . . .	76

Lista de acrónimos y abreviaturas

AA Aprendizaje automático

ACS *American Cancer Society*

bbox *Bounding box*

BLI *Blue Light Imaging*

CCMIJU Centro de Cirugía de Mínima Invasión Jesús Usón

CCR Cáncer colorrectal

CIoU *Complete Intersection over Union*

CLS *Classification*

CNN *Convolutional Neural Network*

CSV *Comma Separated Values*

DeiT *Data-efficient Image Transformer*

FICE *Flexible Imaging Color Enhancement*

FIT *Faecal Immunochemical Test*

gFOBT Prueba de sangre oculta en heces basada en guayacol

IA Inteligencia Artificial

IDP *Image Dataset Processor*

IoU Intersección sobre Unión

JSON *JavaScript Object Notation*

LayerNorm	<i>Layer Normalization</i>
LCI	<i>Linked Color Imaging</i>
LDA	<i>Linear Transformation Techniques</i>
MAE	<i>Mean Absolute Error</i>
MLP	<i>multilayer perceptron</i>
MSE	<i>Mean Square Error</i>
NBI	<i>Narrow-Band Image</i>
NLP	<i>Natural Language Procesing</i>
PCA	<i>Principal Component Análisis</i>
PVT	<i>Pyramid Vision Transformer</i>
SDPA	<i>Scalar Dot Product Attention</i>
SRA	<i>Spatial-Reduction Attention</i>
TFG	<i>Trabajo Fin de Grado</i>
ViT	<i>Visual Transformer</i>
WL	<i>White Light Image</i>

Capítulo 1

Introducción y objetivos

Este capítulo constituye un preámbulo al problema de la eficiencia de la detección del cáncer colorrectal (CCR) aplicando modelos fundacionales, incluyendo la eficiencia de su aplicación y posible mejora respecto a aplicar aproximaciones más tradicionales a este campo. A continuación, se definen los objetivos del presente Trabajo de Fin de Grado (TFG), la correspondiente metodología seguida y una breve estructura de la memoria.

1.1. Contexto y motivación

El CCR es una de las principales causas de mortalidad por cáncer a nivel mundial, representando aproximadamente el 10 % de los casos diagnosticados [3] con una tendencia a aumentar el número de casos. En España tan solo durante el año 2024 se diagnosticaron 44.294 nuevos casos de este tipo de cáncer [4], y siguiendo la línea global, en España se espera que para 2040 incrementen los casos de 150 mil a 207.6 mil casos en hombres y de 111 mil a 133.8 mil casos en mujeres, como indica la Figura 1.1), lo que subraya la importancia de estrategias efectivas de prevención y detección temprana.

La mayoría de los CCRs empiezan como un pequeño crecimiento en el revestimiento interno del colon o el recto, comúnmente pólipos adenomatosos o lesiones benignas, que pueden evolucionar hacia formas malignas si no se detectan y extirpan a tiempo [5]. Para ello, en este campo, la colonoscopia se ha consolidado junto a las pruebas químicas como un estándar en la detección precoz de estas lesiones, permitiendo la visualización directa del colon y la posibilidad de una intervención inmediata para extirpar la lesión.

Sin embargo, la efectividad de la colonoscopia depende en gran medida de la habilidad

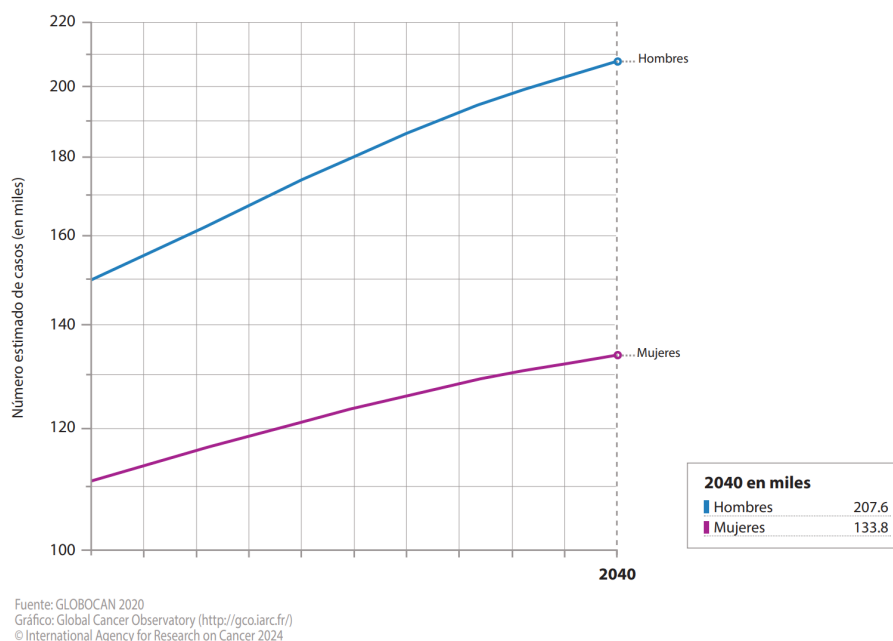


Figura 1.1: Incidencia estimada de cánceres en España para los años 2020 y 2040, por sexos. Imagen obtenida de la *Sociedad Española de Oncología Médica* [4].

y experiencia del endoscopista, así como de la calidad de las imágenes obtenidas [6, 7]. En este contexto, la aplicación de herramientas de inteligencia artificial (IA), tan eficiente en otros campos, ha emergido como una estrategia prometedora para mejorar la precisión y eficiencia en la detección de pólipos durante las colonoscopias.

Estudios como *Detection of Colorectal Polyps from Colonoscopy Using Machine Learning* [8] evidencian el impacto actual de arquitecturas tradicionales, como *SegNet*, que alcanza una precisión del 81.7 %. Asimismo, trabajos como *Focus U-Net: A novel dual attention-gated CNN for polyp segmentation during colonoscopy* [9] reportan una precisión del 80,9 %. Estas arquitecturas más tradicionales de tipo *Convolutional Neural Networks* (CNN) han demostrado resultados clínicamente relevantes y ya se aplican en entornos reales, como lo corrobora el estudio clínico *A prospective multicenter randomized controlled trial on artificial intelligence assisted colonoscopy for enhanced polyp detection* [10], en el que una IA de tipo CNN (*RetinaNet*) fue evaluada en seis centros distintos con un total de 805 pacientes, obteniendo tasas de detección significativamente superiores frente al grupo de control.

Estos resultados refuerzan la evidencia sobre los beneficios de la IA en la detección de pólipos colorrectales. Sin embargo la aplicación de nuevas arquitecturas desarrolladas en los últimos años, como los modelos fundacionales basados en *Vision Transformers* (ViT), sigue siendo un terreno poco explorado en este campo. Estas arquitecturas introducen mecanismos como las ca-

bezas de atención, que han demostrado un rendimiento prometedor en tareas de segmentación y clasificación de imágenes médicas. De hecho, algunos trabajos recientes reportan que superan en ocasiones a modelos basados en redes CNN tradicionales [11, 12]. Estas propuestas emergentes ofrecen el potencial de brindar herramientas más robustas y precisas para el personal médico en la lucha contra el CCR.

1.2. Objetivos

El objetivo principal de este TFG es evaluar la robustez y efectividad de modelos fundacionales basados en ViT para la detección de pólipos en imágenes de colonoscopias. Para ello, se plantean los siguientes objetivos específicos:

- Evaluar la robustez y efectividad de diferentes modelos ViT como: *ViT-large*, *YolosB-BoxOnly*, *ViT-small*, *PVT* y *DeiT*, en la tarea de detección de pólipos en imágenes de colonoscopias.
- Comparar el rendimiento de los modelos ViT con otros enfoques tradicionales de aprendizaje profundo, como es el caso de: *YOLOv8* y *DenseNet*.
- Evaluar la capacidad de generalización de lesiones de distinto tipo, en modelos entrenados con diferentes conjuntos de datos, incluyendo imágenes *WL* y *NBI*.
- Contribuir al desarrollo de herramientas asistidas por IA, ofreciendo una interfaz para obtener y trabajar con *datasets* y modelos en este ámbito.

Así pues, este TFG aporta una herramienta para cargar y analizar *datasets* con pólipos además de manejar el entrenamiento de distintas arquitecturas con estos mismos. Esto permite en última instancia completar el objetivo principal de este TFG: el análisis de distintas arquitecturas ViT en el caso de detección de pólipos en colonoscopias.

1.3. Metodología

Para el desarrollo del TFG se ha utilizado una base de datos de *Piccolo* [13], de acceso bajo solicitud, con 3433, y los *datasets* públicos *CVC* [14] y *PolypDB* [15] con 612 y 3.934 imágenes respectivamente. Estos datos están anonimizados y se presentan en el capítulo 4. Para alcanzar los objetivos descritos anteriormente, se ha llevado a cabo la siguiente metodología:

- **Revisión bibliográfica** exhaustiva de la literatura científica sobre el uso de IA en la detección de pólipos colorrectales, enfocándose en estudios recientes que emplean los *datasets* utilizados durante este estudio así como trabajos enfocados en analizar el rendimiento de modelos ViT y otras arquitecturas aplicadas a imagen en general e imagen médica en particular.
- **Selección de conjuntos de datos adecuados**, como son el caso de los *datasets* de *Piccolo*, *CVC* y *PolypDB*, que contienen imágenes endoscopias con segmentaciones manuales y metadatos clínicos, permitiendo una evaluación detallada de los modelos entrenados además de una garantía de la independencia y integridad de los *splits*.
- **Análisis de la composición de los datos**, así como la naturaleza de las imágenes y información relevante de la composición de las mismas como son la distribución de los pólipos o el volumen de los mismos en la imagen, tanto en general como en cada *split* del conjunto final.
- **Preprocesamiento de datos**, Las imágenes han sido preprocesadas para estandarizar su tamaño, normalizar los valores de píxeles y obtener sus *bounding boxes*(bbox), con el fin de mejorar la efectividad del entrenamiento, proporcionar el método de detección de los pólipos y en ocasiones adaptar la imagen a los requerimientos de las arquitecturas.
- **Entrenamiento de los distintos modelos** tipo ViT además de arquitecturas más tradicionales como CNN para su posterior análisis, creando *benchmarks* adecuados, ajustando hiperparámetros, cabezas de detección, etc.
- **Evaluación del rendimiento** de cada modelo individualmente utilizando métricas estándar como el valor de su función de pérdidas o su Intersección sobre Unión (IoU), así como resultados de análisis más específicos como volumen de las bbox detectadas o su distribución, todo ello tanto en los conjuntos de datos de *train* como en los de *test*.
- **Análisis comparativo** entre las distintas arquitecturas tanto modelos ViT como otros enfoques tradicionales, como CNN, para determinar las ventajas y limitaciones de cada enfoque en la tarea de detección de pólipos.

En la Figura 1.2 se presenta un diagrama de Gantt que detalla las diferentes tareas realizadas durante el presente TFG. Cabe mencionar que es una representación a modo orientativa, y en todo momento se ha intentado establecer el desarrollo temporal de cada tarea con la mayor exactitud posible.

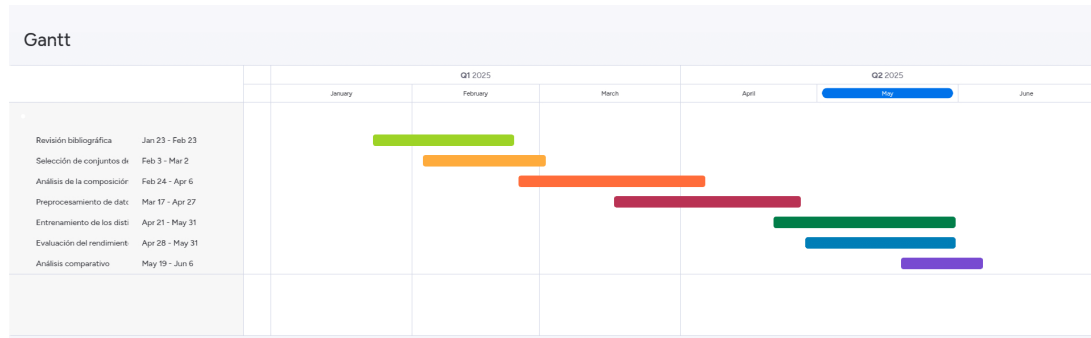


Figura 1.2: Diagrama de Gantt que muestra las tareas realizadas durante el desarrollo del TFG.

1.4. Estructura de la memoria

A continuación se describe el contenido incluido en cada capítulo de la memoria:

- **Capítulo 1: Introducción y objetivos.** Este capítulo presenta el contexto clínico del CCR, destacando la importancia de la detección temprana de pólipos mediante colonoscopias. Se expone la motivación del estudio, centrada en la aplicación de modelos fundacionales de IA llamados ViT para mejorar la detección de pólipos. Además, se definen los objetivos generales y específicos del TFG.
- **Capítulo 2: Conceptos previos.** Se proporciona una revisión de los fundamentos teóricos necesarios para comprender el estudio, incluyendo aspectos básicos sobre el CCR y la colonoscopia, pero especialmente conceptos técnicos entre los que se comprenden: las arquitecturas usadas, métricas usadas, fundamentos generales sobre el entrenamiento o los datos usados.
- **Capítulo 3: Métodos.** Donde repasamos la aplicación práctica del caso de estudio. Este capítulo se compone tanto de detalles sobre el software utilizado y su uso, la utilización de cada una de las herramientas software desarrolladas y utilizadas durante este TFG, hasta la arquitectura de cada modelo utilizado y su adaptación para la detección en el contexto estudiado.
- **Capítulo 4: Base de datos y análisis descriptivo.** En este apartado se describe la base de datos utilizada, detallando el origen de las imágenes de colonoscopias, las características de las anotaciones de pólipos y el proceso de necesario de preprocesamiento de datos. Además se realiza un análisis más exhaustivo y explícito de la composición y detalles de las imágenes incluidas en cada *datasets*.

- **Capítulo 5: Experimentos y resultados.** Este capítulo se presentan y analizan los resultados obtenidos, analizando las arquitecturas estudiadas con distintos conjuntos de datos, y mostrando el rendimiento de cada una de ellas, finalmente comparando el resultado de enfoques más tradicionales con los modelos fundacionales.
- **Capítulo 6: Conclusiones y líneas futuras.** Se resumen los hallazgos más relevantes del estudio, destacando las ventajas y limitaciones de los modelos fundacionales en la detección de pólipos en imágenes de colonoscopias. Además, se discuten las implicaciones clínicas de los resultados y se proponen posibles líneas de investigación futura para mejorar la precisión y aplicabilidad de estos modelos en entornos clínicos reales.

Capítulo 2

Conceptos previos

En este capítulo se introducen una serie de conceptos fundamentales que ayudan a entender el marco clínico y técnico sobre el que se desarrolla este TFG.

2.1. Cáncer colorrectal

El foco del estudio de este TFG se centra en la detección del CCR. Para poder entender mejor esta enfermedad, resulta útil conocer las bases de el funcionamiento del colon y el recto y el desarrollo de esta enfermedad.

El colon y el recto conforman el intestino grueso, el cual forma parte del sistema digestivo, referido también como sistema gastrointestinal, compuesto por un tubo muscular de aproximadamente 1.5 metros de longitud [16]. Sus diferentes segmentos reciben su nombre según la dirección en la que se desplazan los alimentos a través de él, siendo estos: colon ascendente, colon transversal, colon descendente y colon sigmoide. Podemos ver su distribución en la Figura 2.1.

El CCR se origina en esta zona, es decir, en el colon o en el recto, y es uno de los tipos de cáncer más frecuentes a nivel mundial. Solo en el año 2020, ocupó el tercer lugar en número de casos globales, por detrás del cáncer de mama y el de pulmón (ver Figura 2.1). En España, representa el segundo tipo de cáncer más diagnosticado, con 27.009 casos ese mismo año, solo superado por el cáncer de próstata, con 30.316 diagnósticos [4]. A pesar de estas cifras, el CCR suele desarrollarse de forma lenta, lo que ofrece una valiosa oportunidad para detectarlo de forma temprana y prevenir su progresión mediante programas de cribado adecuados.

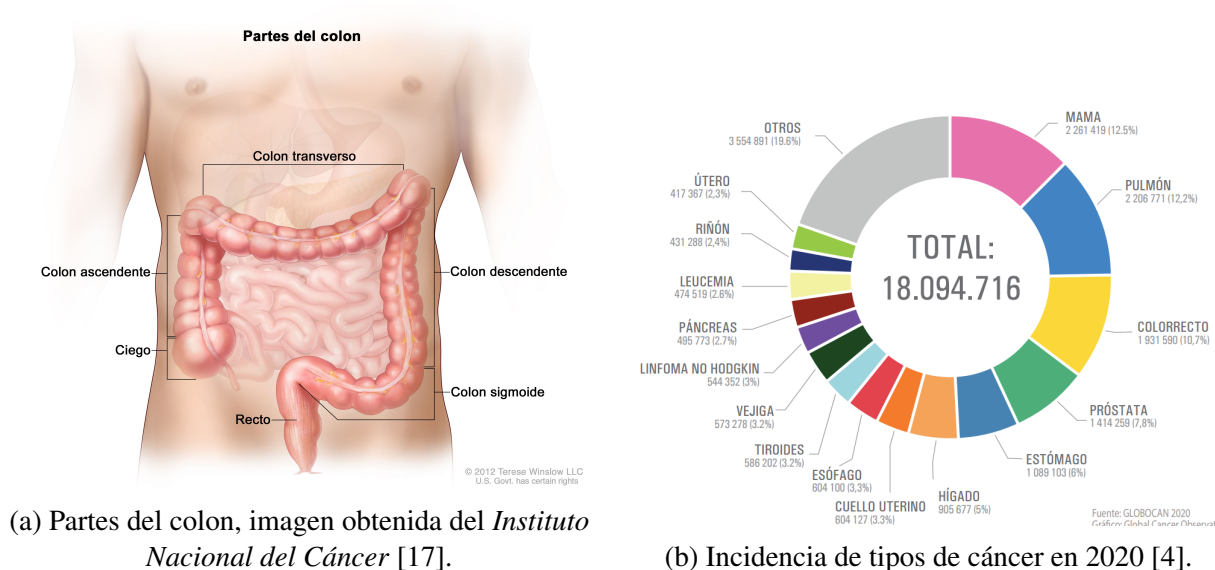


Figura 2.1: Partes del colon e incidencia de tipo de cáncer.

2.1.1. Origen y evolución

El CCR suele comenzar como un crecimiento en el revestimiento interno del cólon o recto desarrollando lesiones precursoras benignas denominadas pólipos. Estas lesiones son consideradas precancerosas o benignas [18]. Con el tiempo algunos de estos pólipos llegan a convertirse en lesiones malignas, pero este proceso, conocido como la secuencia adenoma-carcinoma [19] no es igual para todos los tipos de lesiones, la probabilidad y tiempo requeridos para evolucionar a una lesión maligna depende de la clase de pólipo a la que pertenezca la lesión. Los tipos son los siguientes según la *American Cancer Society (ACS)* [16]:

1. **Pólipos adenomatosos (adenomas):** Estos pólipos algunas veces se transforman en cáncer. Debido a esto, los adenomas se denominan *afecciones precancerosas*. Los 3 tipos de adenomas son tubulares, vellosos y tubulovellosos [16], estos se pueden ver en la Figura 2.2, imágenes empezando por la parte superior izquierda y siguiendo en horizontal, 1ª, 2ª y 3ª imágenes.
2. **Pólipos inflamatorios y pólipos hiperplásicos:** estos pólipos son más frecuentes, pero en general no son precancerosos, podemos verlos en la Figura 2.2, empezando por la parte inferior izquierda y siguiendo en horizontal, imágenes 2ª y 3ª. Algunas personas con pólipos hiperplásicos (más de 1 cm) podrían requerir la realización de una colonoscopia para ser detectado adecuadamente [16].

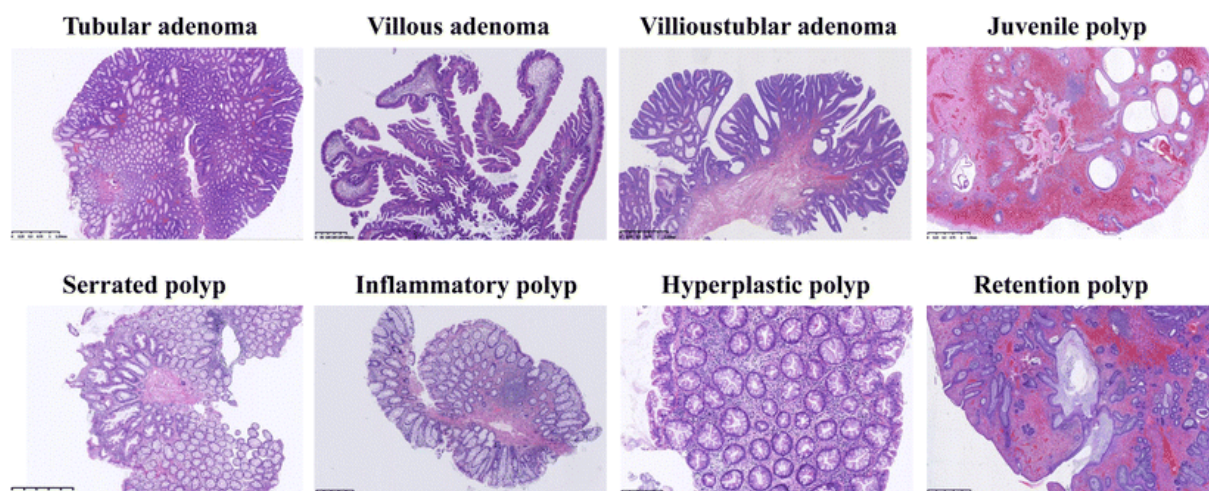


Figura 2.2: Tipos de pólipos bajo microscopio y con tinción con hematoxilina y eosina. Imagen obtenida del *International Journal of Colorectal Disease* [20].

3. **Pólipos serrados sésiles (SSP) y adenomas serrados tradicionales (TSA):** Estos pólipos a menudo son tratados como adenomas ya que tienen una mayor probabilidad de evolucionar a una lesión maligna [16]. Podemos ver una muestra de un pólipo serrado en la imagen inferior izquierda de la Figura 2.2

Naturalmente esta no es la única forma de clasificar las lesiones. En el contexto de este TFG es común la utilización de la clasificación de París, presentada por primera vez en "*The Paris endoscopic classification of superficial neoplastic lesions: esophagus, stomach, and colon*" [21], esta clasificación es parecida a la presentada por la ACS y es utilizada en los datos aportados por el *dataset* de Piccolo [1]. Estas clases podemos verlas en la Figura 2.3.

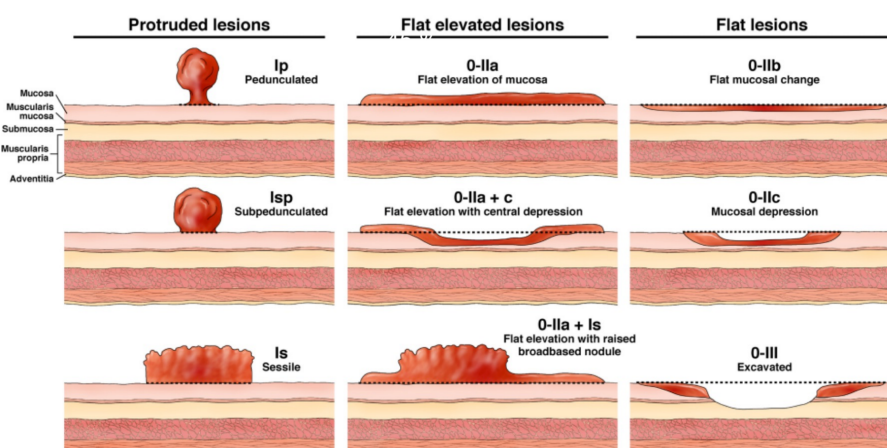


Figura 2.3: Morfología de la clasificación de París. Imagen obtenida de la Sociedad Andaluza de Patología Digestiva [22].

También hay otros factores propios a cada caso que hacen que el riesgo de una lesión maligna sea mayor, como son:

1. Un pólipo de más de un centímetro.
2. Más de 3 pólipos.
3. Tras extirpar un pólipo se descubre una zona del revestimiento del intestino cuyas células tienen un aspecto anormal o precanceroso.

2.1.2. Fases de desarrollo del pólipo durante el CCR

Si finalmente el cáncer no es detectado a tiempo y el cáncer crece dentro del pólipo, este pasa por las siguientes fases de crecimiento o secuencia adenoma-carcinoma Figura 2.5.

1. **Formación de pólipos (Etapa I).** El cáncer se origina en la capa más interna del colon (la mucosa) y crece hacia el exterior a través de algunas o todas las capas del intestino Figura 2.4

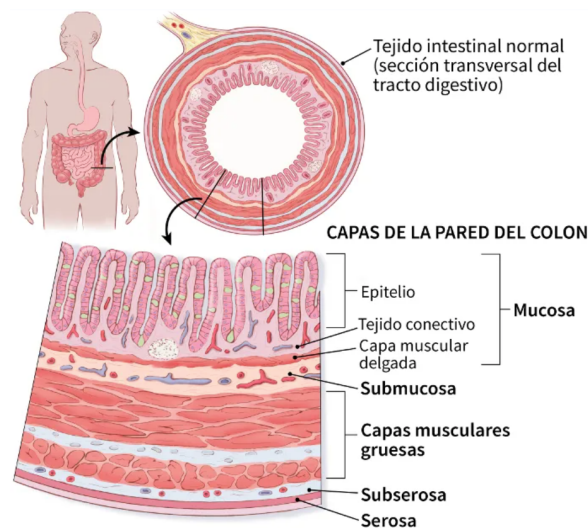


Figura 2.4: Capas del colon. Imagen extraída de la web de la ACS [16].

2. **Crecimiento local (Etapa II).** Si un pólipo se convierte en canceroso, las células anormales comienzan a crecer y a invadir las capas más profundas del intestino e incluso afectando a tejidos cercanos.

3. **Diseminación a ganglios linfáticos (Etapa III).** El adenoma al haber atravesado la pared del colon, tiene acceso a los distintos vasos sanguíneos o los vasos linfáticos (canales diminutos que transportan material de desecho y líquido) y desde ahí se ha desplazado a los ganglios linfáticos.
4. **Metástasis (Etapa IV).** El cáncer se ha diseminado a otras partes del cuerpo, como el hígado, los pulmones o más comúnmente el hígado [23].

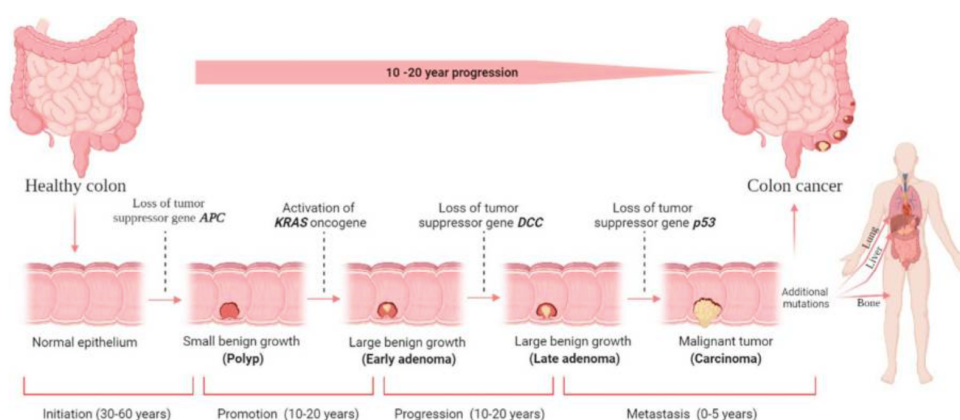


Figura 2.5: Evolución del CCR. Imagen obtenida de [23].

2.1.3. Métodos de detección

En una enfermedad tan extendida y sobre todo con un periodo de evolución tan lento, es esencial hablar sobre las distintas formas de detección, las más comunes:

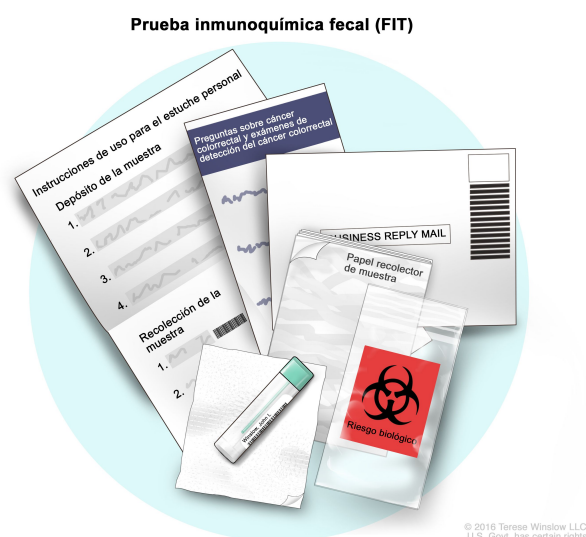
1. **Pruebas de sangre oculta en las heces:** Estas pruebas se basan en detectar sangre oculta en las heces, lo cual puede ser una señal temprana de pólipos o CCR, no puede garantizar los resultados debido a que la sangre podría provenir de otras partes del tracto digestivo, como el estómago. Los métodos más comunes son la *Prueba de sangre oculta en heces basada en guayacol* (gFOBT) y especialmente la *Prueba inmunoquímica fecal* (FIT), prueba que podemos ver en la Figura 2.6. Esta última está muy extendida debido a la facilidad de la misma y comodidad para el paciente, y está probada su eficacia reduciendo en un 34 % la incidencia en un estadio avanzado y su mortalidad en un 40 % [24]. Además a estas dos se le añade la prueba de ADN en heces, que coteja tanto la sangre oculta como alteraciones genéticas en las heces que podrían indicar la presencia de células anormales.
2. **Exámenes visuales:** Estas pruebas se basan en observar y detectar cualquier anomalía en el colon. Aunque se hacen con menos frecuencia que las de heces debido a que requieren

de una preparación mayor y suponen mayor comodidad y riesgo para el paciente, entre ellas se encuentran: la colonografía por CT, la Sigmoidoscopia flexible y la colonoscopia, donde se utilizan herramientas para realizarlas como la COMBO-3 que vemos en la Figura 2.6. La colonoscopia tiene la ventaja de determinar y poder incluso extirpar en el momento el pólipo.

Es debatido la eficacia de estos modelos y un estudio publicado en *The Lancet* hecho a 57.000 pacientes a lo largo de 10 años concluyó que tanto el FIT como las colonoscopias tienen una efectividad parecida [25], teniendo esta última la ventaja de poder intervenir y extirpar la lesión en el momento.



(a)



(b)

Figura 2.6: Imágenes de COMBO-3 y prueba inmunoquímica fecal. (a) COMBO-3 *Gastroscopy and Colonoscopy System* (ENDOMED Systems); (b) Prueba Inmunoquímica Fecal (FIT). Imágenes obtenidas de la web del *National Cancer Institute* [26].

2.2. Aprendizaje automático

El aprendizaje automático (AA), o *machine learning*, constituye una rama fundamental de la IA que se enfoca en el desarrollo de algoritmos capaces de aprender a partir de datos y mejorar su rendimiento con la experiencia, sin necesidad de ser programados explícitamente para cada tarea específica. Este enfoque ha transformado diversos campos, desde la medicina hasta la industria, al permitir la automatización de procesos complejos y la toma de decisiones basada en datos.

En el ámbito de la medicina, y más específicamente en el contexto de la imagen médica relacionada con el CCR, el AA ha demostrado ser una herramienta poderosa para la detección y clasificación de pólipos colorrectales. La capacidad de los modelos de AA para analizar grandes volúmenes de datos y reconocer en estos patrones sutiles, especialmente en imagen médica, ha mejorado significativamente la precisión y eficiencia en el diagnóstico en todos los estadios del CCR [27], contribuyendo a una detección temprana y, por ende, a un mejor pronóstico para los pacientes.

En este apartado se ofrece una visión general de los conceptos clave del AA relevantes para la detección de pólipos mediante técnicas de visión por computador. Se abordarán aspectos como las principales categorías de aprendizaje, vocabulario utilizado durante el entrenamiento, así como una explicación formal en mayor profundidad de las arquitecturas y técnicas usadas. El objetivo es proporcionar al lector una base sólida de los conceptos teóricos en los que se basa y utiliza este TFG, y que así le facilite la comprensión de cómo y porqué se aplican estas técnicas explicada en el capítulo 3.

2.2.1. Definiciones generales

- **Aprendizaje automático:** Esta rama de la IA es definido por Sebastian Raschka como '*la aplicación y ciencia de la algorítmica que da sentido a los datos*' [28] y siendo un poco más extensos podemos definir el AA como; Una rama de la IA que se centra en desarrollar algoritmos y modelos estadísticos que permiten a un ordenador aprender a partir de datos y mejorar su rendimiento en tareas específicas sin ser programadas explícitamente para cada una de ellas.
- **Neurona artificial:** Presentada por primera vez por Warren McCulloch y Walter Pitts [29], la neurona artificial ha evolucionado con el tiempo. Inicialmente concebida como un modelo matemático inspirado en la neurona biológica, McCulloch y Pitts la

describen como *una simple puerta lógica con salida binaria; múltiples señales de entrada y si se supera un cierto umbral se produce una salida* [28]. Modelos posteriores como el *Perceptron* de Frank Rosenblatt [30] o el *ADaptive LInear NEuron (Adaline)* de Barnard Widrow [31], mejoraron y ampliaron la capacidad de aprendizaje de una neurona artificial.

Siguiendo *Machine Learning with PyTorch and Scikit-Learn* [28] podemos dar una definición más formal de una neurona artificial en su versión más sencilla, la clasificación binaria (clase 1 o 0): Definimos una matriz de elementos de entrada x , y una matriz de pesos o "importancia" que damos a esos datos de entrada W que es igual a la entrada neta $z = w_1x_1 + w_2x_2 + \dots + w_mx_m$

$$W = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}, \quad x = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$$

Ahora si una entrada en particular x^i es mayor a un cierto θ entonces la salida predicha será la clase 1 y si no la 0. Esto se define como una función de decisión básica:

$$\sigma(z) = \begin{cases} 1 & \text{si } z \geq \theta \\ 0 & \text{resto} \end{cases}$$

A partir de esta definición y sus mejoras lógicas, se estructuran todas las neuronas utilizadas como el bloque más sencillo para construir cualquier red neuronal.

- **Red Neuronal:** Siguiendo la definición de IBM [32], "*Una red neuronal es un programa o modelo de AA que toma decisiones de manera similar al cerebro humano, mediante el uso de procesos que imitan la forma en que las neuronas biológicas trabajan juntas para identificar fenómenos, sopesar opciones y llegar a conclusiones*". Es decir, las redes neuronales son un conjunto de neuronas artificiales organizadas generalmente en capas y conectadas entre si. Para entender mejor como funciona explicaré las partes de la red neuronal usando una de las redes neuronales más comunes y sencillas, la *multilayer perceptron* (MLP), cuya estructura podemos ver en la Figura 2.6a.

Una red neuronal MLP es aquella en cuyas capas están completamente conectadas entre si. la capa más a la izquierda en amarillo es **la capa de entrada**, representa como la red "lee" los datos, en el caso de datos tabulares, cada una de las columnas o "*características*" de cada muestra es una entrada, en el caso de una imagen, cada píxel de la misma.

Entre la capa de entrada y la capa de salida, se encuentran **las capas ocultas**, estas capas están interconectadas entre sí y compuestas por *neuronas artificiales* (pág. 13). Cada una de estas neuronas realiza cálculos utilizando de entrada cada una de las neuronas de la capa anterior, multiplicando estas por los pesos correspondientes, siendo su versión más sencilla la explicada en el punto anterior.

Finalmente este aprendizaje y procesamiento de la entrada a lo largo de las capas ocultas, converge finalmente en **la capa de salida**, la cuál interpreta las la salida de la última capa dando el resultado. En este ejemplo si la entrada es de clase A o B. En este TFG obtenemos una *bbox* indicando el pólipo en la imagen.

Todo este proceso permite a una red neuronal aprender patrones complejos con el suficiente entrenamiento. Mencionar que una red neuronal contiene más de una capa, como es el caso de la Figura 2.6a se denomina una red profunda o "*Deep learning*".

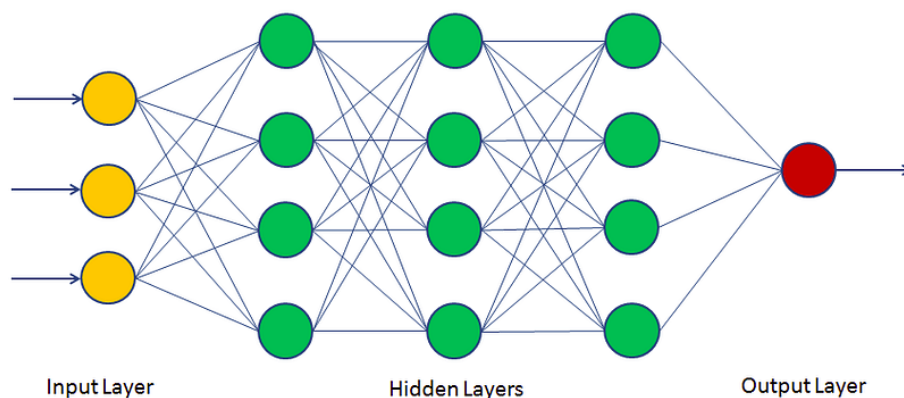


Figura 2.7: Arquitectura simplificada de una MLP. Imagen obtenida de [33].

- **Modelos fundacionales:** Según "*On the Opportunities and Risks of Foundation Models*" [34], son modelos de IA entrenados con enormes volúmenes de datos no etiquetados mediante aprendizaje auto-supervisado. Estos modelos, como *BERT*, *GPT-3* o *DALL·E*, se caracterizan por su capacidad para adaptarse a una amplia gama de tareas sin necesidad de ser diseñados específicamente para cada una. Es decir, estos modelos, gracias a su complejidad y su entrenamiento a gran escala, les permite adquirir representaciones generales del lenguaje, imágenes u otros tipos de datos, para luego poder ajustarse a aplicaciones particulares mediante técnicas como el ajuste fino "*fine-tuning*" (pág. 17). Este enfoque ha transformado el desarrollo de la IA, facilitando la creación de aplicaciones más rápidas y eficientes, y es el enfoque utilizado para este TFG, ya que usando modelos fundacionales pre-entrenados, podemos adaptarlos con un coste computacional menor, necesario para

los recursos de este TFG, y así obtener resultados prometedores en detección.

- **Transformers:** Estos son modelos de aprendizaje profundo diseñados para procesar secuencias de datos, como texto o imágenes, utilizando *mecanismos de atención*, estos permiten que el modelo enfoque diferentes partes de la entrada de manera dinámica. Explicamos en mayor profundidad esta arquitectura en su aplicación a imagen, conocida como *ViT* ya que es la arquitectura estudiada en este TFG.

2.2.2. Tipos de aprendizaje automático

1. **Aprendizaje supervisado:** Esta técnica de aprendizaje es fundamental y la más extendida en el campo del AA. Consiste en entrenar un modelo utilizando un conjunto de datos etiquetado. Esto significa que cada entrada en el conjunto de datos va acompañada de una salida objetivo o etiqueta.

A partir de estas etiquetas el proceso de entrenamiento consiste en alimentar el modelo con los datos y comparando su salida con la etiqueta correspondiente, al comparar salida y etiqueta podemos obtener el error a través de una función de pérdida y entrenar el modelo usando *back-propagation*.

El objetivo del modelo es aprender una función que, a partir de las entradas, pueda predecir correctamente las salidas correspondientes. Este proceso implica ajustar los parámetros del modelo para minimizar la diferencia entre las predicciones del modelo y las etiquetas reales, utilizando una función de pérdida adecuada.

2. **Aprendizaje no supervisado:** Este tipo de aprendizaje es una rama del AA que se enfoca en descubrir estructuras ocultas en conjuntos de datos sin etiquetas. Es decir, a diferencia del aprendizaje supervisado, donde los modelos se entrenan con datos etiquetados y se puede comparar la salida del modelo con una salida objetivo, en el aprendizaje no supervisado se trabaja con datos sin etiquetar, buscando patrones o agrupaciones inherentes, haciendo posible explorar la estructura de los datos para extraer información significativa sin la guía de una variable de salida conocida o una función de recompensa, como en el caso del aprendizaje por refuerzo.

Entre las técnicas comunes se encuentran el *clustering*, que organiza los datos en subgrupos significativos basados en similitudes o técnicas de reducción de dimensionalidad como *principal component análisis* (PCA) [28] o *linear transformation techniques* (LDA) [28], que simplifica los datos de alta dimensión manteniendo la información esen-

cial. Estas técnicas son fundamentales para tareas como la segmentación de clientes o la compresión de datos.

2.2.3. Técnicas del aprendizaje automático

1. **Preprocesado:** Esta suele ser la primera fase del entrenamiento y es fundamental en cualquier aplicación de AA, ya que transforma los datos brutos de cualquier *dataset* en un formato limpio y estructurado, adecuado para el entrenamiento del modelo. Este proceso implica varias tareas clave que dependen de la naturaleza de los datos, suelen realizarse:
 - a) **Limpieza de datos** para manejar valores faltantes y eliminar inconsistencias
 - b) **transformación de variables categóricas**, o variables sin sentido numérico, mediante técnicas de codificación como *one-hot encoding* [28]
 - c) **Normalización o estandarización** de características numéricas para asegurar que todas las variables contribuyan equitativamente al modelo.
 - d) **Detección y tratamiento de valores atípicos**. Estos valores podrían distorsionar los resultados además de hacer el entrenamiento más inestable y deficiente.

Todas estas técnicas son parte de uno los pasos más importantes, si no el que más, para una aplicación exitosa de IA, ya que un preprocesamiento adecuado mejora la calidad de los datos, lo que se traduce en modelos más precisos y generalizables, capaces de ofrecer mejores predicciones.

2. **Fine-tuning:** Esta una técnica es fundamental en los experimentos de este TFG. Consiste en tomar un modelo previamente entrenado en un conjunto de datos amplio y general, y adaptarlo a una tarea específica utilizando un conjunto de datos más pequeño y especializado [35].

El proceso implica ajustar los pesos del modelo pre-entrenado mediante entrenamiento adicional con datos etiquetados de la tarea objetivo, permitiendo que el modelo conserve el conocimiento general adquirido durante el pre-entrenamiento y lo refine para desempeñarse eficazmente en la nueva tarea. Este enfoque es especialmente útil cuando se dispone de recursos computacionales limitados o conjuntos de datos pequeños, ya que evita entrenar un modelo desde cero y aprovecha el aprendizaje previo para lograr una adaptación eficiente y precisa.

3. **Segmentación:** La segmentación consiste en dividir una imagen digital en regiones o "segmentos", agrupando píxeles con características similares como color, textura o intensidad. Con ello permite al modelo identificar y analizar componentes específicos dentro de una imagen como bordes y regiones de fondo, facilitando tareas como la detección de objetos, el reconocimiento de patrones y la comprensión de escenas complejas [36]. Por ejemplo, en aplicaciones médicas, la segmentación es crucial para delinear estructuras anatómicas o identificar áreas afectadas en imágenes de diagnóstico. En el contexto de vehículos autónomos, ayuda a distinguir elementos como peatones, señales de tráfico y carriles en la carretera. Estas técnicas son esenciales para que los sistemas de IA interpreten y actúen sobre datos visuales de manera precisa y eficiente.
4. **Detección:** La detección de objetos en el contexto de la visión artificial, es una técnica que permite a los sistemas identificar y localizar objetos específicos dentro de imágenes o secuencias de vídeo. Esta tarea consta de dos procesos fundamentales [37]:
 - a) **Clasificación**, que determina la categoría del objeto (como persona, vehículo o animal).
 - b) **Localización**, que establece la posición del objeto mediante cuadros delimitadores o *bboxes*.

Estas dos herramientas pueden usarse por separado o combinarlas. La detección es una herramienta fundamental en aplicaciones como; la conducción autónoma, la vigilancia y la seguridad hasta la medicina y la agricultura.

2.2.4. Conjunto de *train*, *validation* y *test*

En esta sección explicamos conceptos relacionados con los datos de *train* y conceptos relacionados.

- **Dataset:** *Dataset* se refiere a una colección de datos, generalmente organizada en tablas, matrices o archivos CSV y JSON, que facilitan su interpretación y análisis de cara a ser usados como datos para entrenar un modelo. Los *datasets* son fundamentales para aplicaciones de análisis de datos, AA e IA, ya que proporcionan datos fiables y accesibles para estos procesos, gracias a que en muchos casos son de acceso público o de fácil acceso para aplicaciones educativas.

Estos conjuntos de datos pueden estar organizados de dos maneras principalmente:

1. **Estructurados.** En estos los datos se encuentran organizados en filas y columnas, generalmente en CSV o JSON.

2. **No estructurados.** Esto suele incluir *datasets* con formato de texto, imagen o audio.

Una organización adecuada de los datos en *datasets* permite a las organizaciones extraer datos relevantes de grandes volúmenes de información recopilada desde diversas fuentes, siendo de esta forma esencial estudiar la naturaleza de estos datos y seleccionar *datasets* adecuados para la aplicación deseada como evaluamos en mayor profundidad en 4.

■ **Splits:** Estos son los distintos subconjuntos de datos en los que se divide un *dataset* completo, es una práctica esencial para poder evaluar y mejorar el rendimiento del modelo. Generalmente se dividen en tres conjuntos:

1. **Conjunto de entrenamiento (*training set*):** El *split* con más cantidad de datos. Se utiliza durante el proceso de entrenamiento y con estos datos se ajustan los pesos del modelo.

2. **Conjunto de validación (*validation set*):** Sirve principalmente para evaluar el rendimiento del modelo ante datos nuevos durante el entrenamiento, también para afinar los hiperparámetros y tomar decisiones sobre la arquitectura del modelo. Es un *split* esencial para poder evaluar y corregir problemas del entrenamiento como el *over-fitting*.

3. **Conjunto de prueba (*test set*):** Se emplea exclusivamente para evaluar el rendimiento final del modelo en datos no vistos durante el entrenamiento y la *validation*. Generalmente el dato del rendimiento final de un modelo se obtiene del rendimiento con este conjunto.

Es importante asegurarse de que la división en *splits* sea apropiada, es decir, que garantiza la independencia y proporción adecuadas para cada *split*. Las proporciones adecuadas varían respecto a la cantidad de datos disponibles, pero una buena práctica es una división 75%, 25% y 25% como podemos verlo en la Figura 2.8. Evitando datos duplicados o datos relacionados en distintos *splits* ya que esto podría dar una falsa impresión de un alto rendimiento del modelo. Por ejemplo, si el conjunto de prueba contiene ejemplos que también están en el conjunto de *train*, el modelo podría simplemente memorizar estos casos en lugar de aprender a generalizar, lo que llevaría a una evaluación engañosa de su desempeño y negaría la principal función de dividir el *dataset*, que es, entrenar y evaluar el modelo apropiadamente.

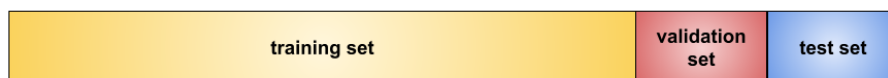


Figura 2.8: *Splits* de un *dataset* bien estructurado. Imagen obtenida de [38].

- **Bounding boxes (bbox):** En visión artificial, una bbox es un recuadro trazado alrededor de un objeto de interés dentro de una imagen. Esta se utiliza principalmente en tareas de detección de objetos, donde es esencial identificar su ubicación en la imagen. Las bboxes se definen típicamente por las coordenadas de sus esquinas superior izquierda e inferior derecha, formato que llamamos durante este TFG "*double-corner*" Figura 2.9a usado en métricas como IoU, otras son; el centro del rectángulo junto con su ancho y alto o "*center*" usado en *YOLO* y esquina superior izquierda junto a anchura y altura de la bbox o "*corner*" usada en *COCO*, entre otras.

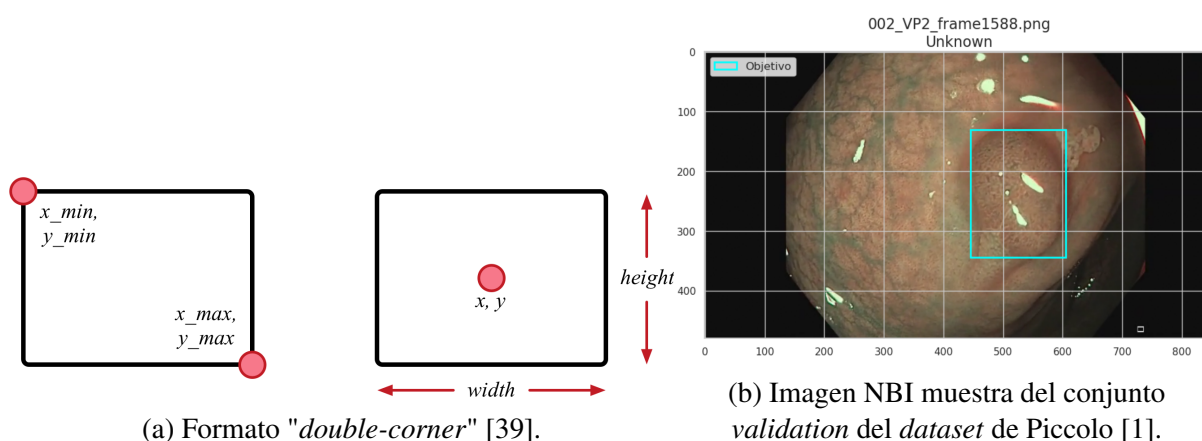


Figura 2.9: Ejemplos de formatos de cajas delimitadoras.

- **Máscaras de segmentación:** En el contexto de la detección de pólipos mediante visión artificial, las máscaras son imágenes binarias que indican la ubicación precisa de los pólipos en imágenes médicas, en concreto las obtenidas por colonoscopia. Estas máscaras forman generalmente parte de los *datasets* más usados 4, y en ellas cada píxel de la imagen se clasifica como perteneciente al pólipo (representado en blanco) o al fondo (en negro) Figura 2.10 Este enfoque permite una identificación detallada de los límites y la forma de los pólipos, facilitando su análisis y diagnóstico.

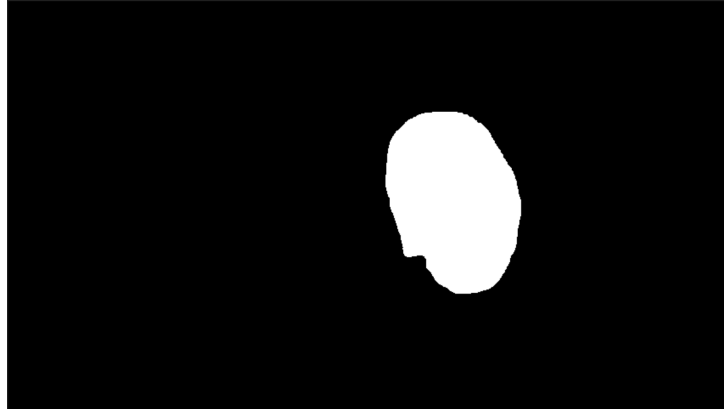


Figura 2.10: Máscara correspondiente a la muestra de la Fig.2.9b del *dataset* Piccolo [1].

- **Escalado de características:** Esta es una técnica de preprocesamiento de datos que ajusta las características numéricas a una misma escala para que sean comparables entre sí. Esto es esencial porque muchos algoritmos de AA, como la regresión logística, las redes neuronales o las máquinas de soporte vectorial, son sensibles a la escala de las características y pueden verse afectados negativamente si algunas variables están en rangos de mayor magnitud respecto a otras.

Existen distintas aproximaciones para aplicar el escalado:

1. **Normalización:** en concreto la técnica "*escalado min-max*", consiste en la transformación de los valores de una característica para que estén dentro de un rango específico, generalmente entre 0 y 1, a través de la fórmula [28]:

$$x_{norm}^{(i)} = \frac{x^{(i)} - x_{min}}{x_{max} - x_{min}}$$

Siendo $x^{(i)}$ el valor a normalizar y x_{min} , x_{max} el mayor y menor valor de la característica. Es una estrategia muy útil cuando se desea mantener la forma de la distribución original de los datos.

2. **Estandarización:** Esta técnica consiste en transformar los datos para que tengan una media de 0 y una desviación estándar de 1. La fórmula es:

$$x_{std}^{(i)} = \frac{x^{(i)} - \mu_x}{\sigma_x} \quad / \quad \sigma_x = \sqrt{\frac{\sum_n (x_n - \mu_x)^2}{n}}$$

Representando μ_x la media de las muestras de una característica, y σ_x su desviación estándar.

Esto hace que los pesos sean mucho más sencillos de aprender además que no cambia la distribución de los datos manteniendo proporciones y información relevante sobre datos anómalos etc, siendo una herramienta ideal para tratar los datos.

Estas técnicas de normalización ayuda a mejorar el rendimiento y la convergencia de los algoritmos de AA, asegurando que cada característica contribuya equitativamente al modelo.

2.2.5. Métricas y Evaluación de los modelos

1. **Precisión de un modelo o Accuracy:** En "*Machine Learning with PyTorch and Scikit-Learn*" Sebastian Raschka se limita a definir el "Accuracy".^{en} el contexto de una aplicación de clasificación como "*la proporción de instancias clasificadas correctamente*" [28]. Lo que quiere decir es que es el término en el que generalmente al acierto o rendimiento de un modelo en una aplicación concreta. Matemáticamente podríamos definirlo como:

$$Accuracy = \frac{N_{\text{mero de predicciones correctas}}}{N_{\text{mero total de correctas}}}$$

Es importante tener en cuenta que, aunque el *accuracy* es una métrica intuitiva y fácil de interpretar, puede no ser adecuada dependiendo de la aplicación. En tales casos, otras métricas como la precisión, la sensibilidad o la puntuación F1 o en el caso de este TFG el IoU.

2. **Intersection over Union:** En el contexto de la visión artificial, en concreto el uso de el IoU es una métrica fundamental para evaluar la precisión de modelos de detección y segmentación de objetos y será la utilizada para evaluar los modelos durante la fase de experimentación de este TFG. Esta métrica cuantifica la superposición entre la predicción del modelo y la etiqueta o salida objetivo, proporcionando una medida de qué tan bien el modelo ha identificado y localizado un objeto en una imagen.

Formalmente podemos definir el IoU para la detección con bboxes como el cociente entre el área de intersección y el área de unión de dos regiones, tal que:

- a) La **intersección** es el área común entre la bbox predicha por el modelo y la caja delimitadora real.
- b) La **unión** es el área total cubierta por ambas cajas, es decir, la suma de sus áreas individuales menos el área de intersección.

$$\text{IoU} = \frac{\text{Área de intersección}}{\text{Área de unión}}$$

El valor del IoU varía entre 1 y 0. Significando 0 que no hay superposición entre las cajas y por lo tanto 1 una coincidencia perfecta de la imagen como se puede apreciar en la Figura 2.11.

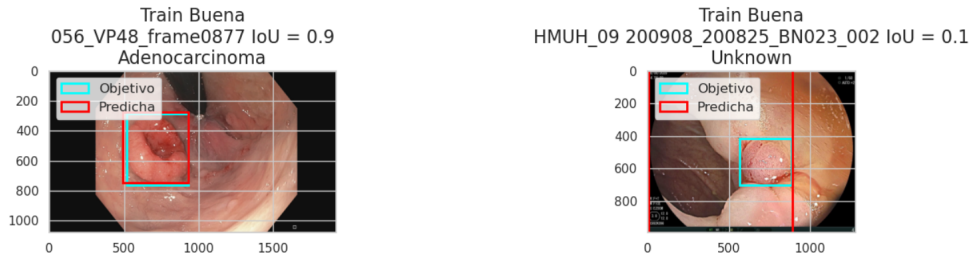


Figura 2.11: Ejemplo de una buena y mala IoU en el conjunto de *train* para el modelo *ViT-Large*.

Cabe añadir que ya que se refiere a un área de coincidencia de las bboxes, consideraremos un muy buen resultado a partir del rango de 0.8 y uno aceptable mente bueno a partir de un 0.6 de valores de IoU.

3. **Función de pérdida o *loss-function*** La función de pérdida (o loss function) es una de las herramientas más cruciales para la aplicación de un modelo. Esta función es un modelo matemático que cuantifica la discrepancia entre las predicciones del modelo y las etiquetas o salida objetivo. Sirve como guía para ajustar los parámetros del modelo durante el proceso de entrenamiento, con el objetivo de minimizar esta discrepancia y mejorar la precisión del modelo, por lo que su elección y apropiado ajuste de sus hiperparámetros es esencial para un entrenamiento efectivo y eficiente.

Existen diversas funciones de pérdida, cada una adecuada para diferentes tipos de problemas, las más comunes son:

- a) **Pérdida cuadrática media (MSE):** Utilizada comúnmente en problemas de regresión y usada como base de funciones de pérdida más complejas, calcula el promedio de los cuadrados de las diferencias entre las predicciones y los valores reales.

Una definición más formal sería [28], dado una salida del modelo $\sigma(z^{(i)})$ y una salida objetivo $y^{(i)}$, podemos definir MSE como:

$$\text{MSE} = \sum_i \frac{1}{2} (\sigma(z^{(i)}) - y^{(i)})^2$$

- b) **Error absoluto Medio (MAE) o función de pérdida L1:** Esta métrica es muy similar a MSE, su diferencia es que se calcula como el promedio de las diferencias absolutas entre las predicciones y los valores reales, esto hace que sea una función más robusta ya que no penaliza en exceso los errores grandes y es fácilmente interpretable ya que está en la misma magnitud que la entrada. Formalmente se define como [40]:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |\sigma(z^{(i)}) - y^{(i)}|$$

Siendo $\sigma(z^{(i)})$ la salida del modelo y $y^{(i)}$ la salida objetivo. Mencionar que esta función de pérdida es la usada durante la primera etapa de experimentación dado que es usada para comparar los vectores de salida y objetivo del modelo.

Estas no son las únicas funciones de pérdida, y hay tantas como aplicaciones de IA diferente, y de hecho es necesario elegir adecuadamente respecto a nuestro problema.

Como cabe esperar si la métrica de precisión usada es IoU, entonces la *loss function* podría ser esta. El problema reside en que IoU es un valor visual y no representa fielmente el error entre los vectores de salida y de entrada, para ello lo lógico es utilizar MAE o mejor aún MSE o L1.

Estas funciones de pérdida sin duda son útiles y funcionan, como describo más adelante. Pero finalmente la función más completa y descriptiva del error en el contexto de este TFG es CIOU. Esto se debe a que combina óptimamente la métrica objetivo IoU con una comprensión de las coordenadas y la distancia euclídea entre los valores de salida y la salida esperada tanto de sus centros como la coherencia de sus proporciones [41].

Esta función de pérdida la hemos escogido ya que es la más completa de las distintas *loss function* probadas y la mejor de la familia de las *IoU* como podemos observar en la tabla aportada por *Prakash Chandra* en su artículo "*IoU Loss Functions for Faster & More Accurate Object Detection*" [41] en la que compara YOLOv3 con el *dataset* de COCO usando las distintas funciones de pérdida Figura 2.12.

Loss / Evaluation	AP	AP75	APsmall	APmedium	APlarge
$\mathcal{L}_{IoU}$	37.93	40.79	21.58	40.82	50.14
$\mathcal{L}_{GIoU}$	38.02	41.11	21.45	41.06	50.21
Relative improv. %	0.24%	0.78%	-0.60%	0.59%	0.14%
$\mathcal{L}_{DIOU}$	38.09	41.11	21.66	41.18	50.32
Relative improv. %	0.42%	0.78%	0.31%	0.88%	0.36%
$\mathcal{L}_{CIOU}$	38.65	41.96	21.32	41.83	51.51
Relative improv. %	1.90%	2.87%	-1.20%	2.47%	2.73%
$\mathcal{L}_{CIOU}(D)$	38.71	42.07	21.37	41.93	51.60
Relative improv. %	2.06%	3.14%	-0.97%	2.72%	2.91%

Figura 2.12: Eficiencia de las distintas métricas de la familia IoU. Datos extraídos de [41].

Una explicación más formal de su funcionamiento se hace durante la explicación de las etapas de entrenamiento 2.3.2.

4. **Optimizador:** Esto se refiere al algoritmo que ajusta iterativamente los parámetros de un modelo, como los pesos y sesgos de una red neuronal, con el objetivo de minimizar una función de pérdida durante el entrenamiento. Esta función es fundamental ya que una vez obtenida la diferencia entre las predicciones del modelo y los valores reales a través de la loss function, es la encargada de actualizar los pesos y buscar con ello minimizar error.

Para entender la función del optimizador seguiremos el funcionamiento de una de los optimizadores más simples, el *descenso por gradiente* y la explicación de este dada por Sebastian Raschka en el libro *Machine Learning with PyTorch and Scikit-Learn* [28].

De forma intuitiva el descenso por gradiente utiliza las derivadas parciales para "descender" por la función como vemos en Fig.2.13. Una forma más formal de definir esta función, dada la *loss function* $L(\mathbf{w}, b)$ y el optimizador definido como esos "pasos" puestos al gradiente $\nabla L(\mathbf{w}, b)$.

$$\mathbf{w} := \mathbf{w} + \Delta \mathbf{w}, \quad b := b + \Delta b$$

Tal que el cambio en los parámetros $\Delta \mathbf{w}$ y Δb se define como el gradiente negativo multiplicada por la tasa de aprendizaje η , esta podemos entenderla como la longitud de los "pasos" por el gradiente.

$$\Delta \mathbf{w} = -\eta \nabla_{\mathbf{w}} L(\mathbf{w}, b), \quad \Delta b = -\eta \nabla_b L(\mathbf{w}, b)$$

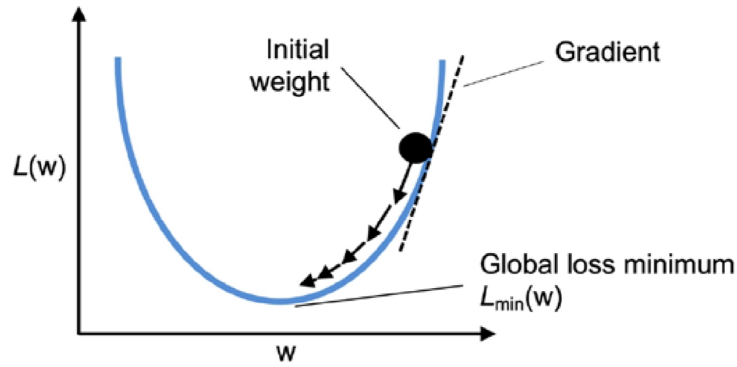


Figura 2.13: Esquema simplificado del descenso por gradiente. Imagen obtenida de [28].

Para calcular el gradiente de la función de pérdida debemos calcular las derivadas parciales de la función de pérdida respecto a cada peso w_j y al *bias* b tal que:

$$\frac{\partial L}{\partial w_j} = -\frac{2}{n} \sum_i (y^{(i)} - \sigma(z^{(i)})) x_j^{(i)}, \quad \frac{\partial L}{\partial b} = -\frac{2}{n} \sum_i (y^{(i)} - \sigma(z^{(i)}))$$

Puntualizar que el 2 en el numerado es una constante de escalado que se puede omitir pero hace más eficiente las operaciones. De esta forma podemos definir la actualización de pesos como:

$$\Delta w_j = -\eta \frac{\partial L}{\partial w_j} \quad \text{y} \quad \Delta b = -\eta \frac{\partial L}{\partial b}$$

Lógicamente los optimizadores al ser la pieza encargada de actualizar los pesos es objeto de mucho estudio y avances con Optimizadores mucho más eficientes y complejos.

En el marco de este TFG utilizamos uno de los más eficientes y probados, el optimizador Adam presentado por *Bernard Widrow* y *Ted Hoff* en la Universidad de Stanford en 1960 [42]. Intuitivamente podemos explicar que Adam se compone de una red neuronal de una sola capa que usa la regla de aprendizaje de mínimos cuadrados para ajustar los pesos.

Adam es una elección adecuada debido a su eficiencia y aproximación sólida probada en múltiples aplicaciones, *Rosie Zhao* hace un análisis de Adam y da una serie de estudios de su buen rendimiento en modelos fundacionales aplicados a visión en "apéndice A y trabajos relacionados" [43].

5. **Overfitting / Underfitting:** Ambos conceptos están relacionados con la capacidad de un modelo de aprender y sobre todo de generalizar con datos nuevos, podemos definir

estos siguiendo la explicación dada por *IBM* [44] como:

- a) **Overfitting** o sobreajuste, ocurre cuando un modelo aprende demasiado bien los datos de entrenamiento, incluyendo el ruido y las anomalías, lo que resulta en un rendimiento deficiente al enfrentar nuevos datos. Esto se debe a que el modelo se adapta excesivamente a los datos específicos de entrenamiento y no generaliza adecuadamente. Podemos ver esta tendencia en la Figura 2.14 ya que el modelo se ajusta a los datos de entrenamiento un 80 y tan solo un 58 para *validation*. Este ha sido un problema recurrente durante este TFG.

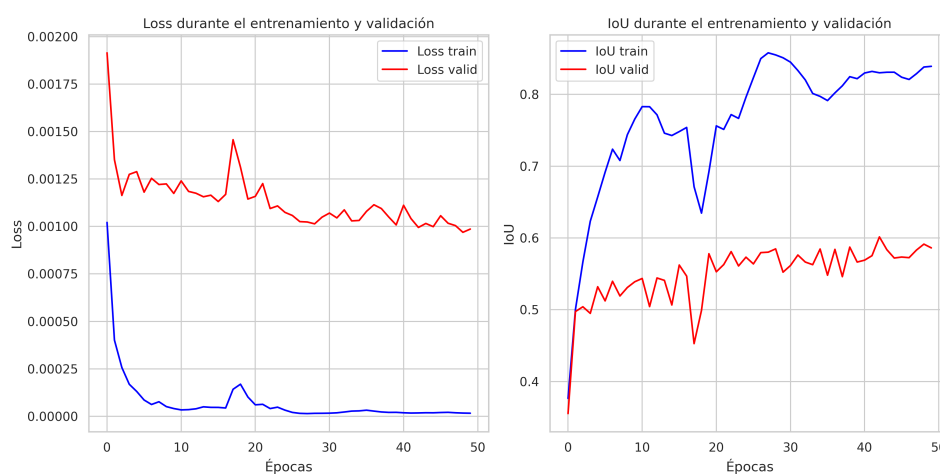


Figura 2.14: Ejemplo de curvas de aprendizaje que presentan *overfitting*.

- b) **Underfitting** o subajuste, sucede cuando el modelo es demasiado simple para capturar los patrones subyacentes de los datos, lo que lleva a un rendimiento pobre tanto en los datos de entrenamiento como en los de prueba. En este caso, el modelo no logra aprender lo suficiente de los datos para hacer predicciones precisas.

El objetivo es encontrar un equilibrio entre estos dos extremos, logrando que el modelo generalice bien a datos no vistos previamente.

2.2.6. Arquitecturas y tecnologías específicas

En este capítulo se describe la tecnología y estructura de las arquitecturas estudiadas en este trabajo.

1. **Tokenización:** Este es el proceso de dividir la entrada en unidades más pequeñas llamadas *tokens* para ser interpretados por el modelo. Esta es una parte esencial en el procesamiento de los datos para formar parte del *input*, es utilizada en contextos donde los datos son muy extensos, como es en procesamiento de textos con técnicas como *word stemming* [28]. En el contexto de la imagen es vital para poder desglosar la matriz de píxeles en una entrada legible y interpretable con la ayuda de los *embeddings*.
2. **Embeddings:** Los *embeddings* son **representaciones numéricas que transforman elementos complejos** como palabras, frases o imágenes en vectores dentro de un espacio multidimensional. Es decir, a una entrada dada, por ejemplo una frase de un texto, se le asigna una "etiqueta" que la sitúa en un espacio multidimensional. Esta transformación permite que las máquinas procesen y comprendan datos de manera más eficiente, ya que esta disposición en un espacio captura las relaciones semánticas y contextuales que existen en este caso entre palabras. Esto resulta en coincidencias interesantes y muy ilustrativas. En el ejemplo de procesamiento de lenguaje natural, palabras con significados similares, como rey y reina, se ubican con una relación parecida a palabras como "hombre" y "mujer" (Figura 2.15). Esta es una herramienta fundamental para que los modelos entiendan elementos abstractos, en este ejemplo en el habla.

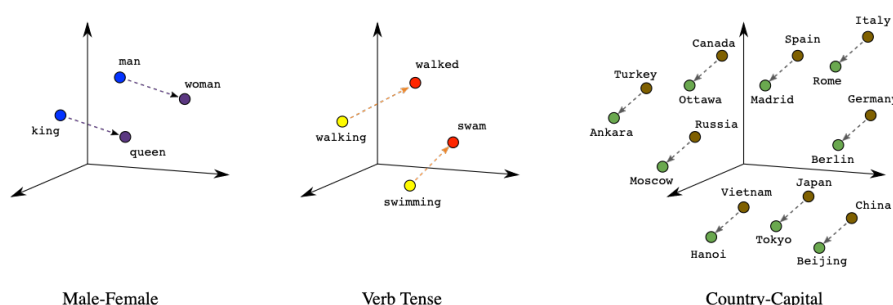


Figura 2.15: Ejemplo de *embeddings* para lenguaje natural [2].

3. **Cabezas de atención (Attention Heads):** Este mecanismo permite que el modelo enfoque simultáneamente diferentes partes de la secuencia de entrada, capturando diversas relaciones contextuales, para ello utiliza una serie de pesos y operaciones relacionando entre sí cada entrada. En una forma abstracta podemos entender este mecanismo como la forma que tiene la IA de que dada una palabra en mitad de una frase que ha de entender, cuan importante o cuánta relación tiene con el resto de palabras de la frase dada como *input*.

Para poder dar una explicación más formal y entender este mecanismo en profundidad seguiré el apartado "*Starting with a basic form of self-attention*" [28]. Contextualicemos el problema asumiendo que tenemos una entrada de longitud T $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$ y una salida $\mathbf{z}^{(1)}, \dots, \mathbf{z}^{(T)}$, cada i elemento de estas secuencias $\mathbf{x}^i$ y $\mathbf{z}^i$ son vectores de longitud $d / \mathbf{x}^{(i)} \in \mathbb{R}$. Ya que el objetivo de la cabeza de atención y del "*self-attention*"^{es} modelar la dependencia del *token* o elemento respecto al resto del *input*, este proceso se obtiene a partir de la siguiente ecuación:

$$\mathbf{z}^{(i)} = \sum_{j=1}^T \alpha_{ij} \mathbf{x}^{(j)}$$

Esta se divide en 3 fases:

- a) **Obtener la importancia del elemento ω_{ij}** basándonos en cómo de similar es al resto de elementos. Esto se consigue haciendo el producto escalar del *token* estudiado o $\mathbf{x}^{(i)}$ con el *token* $\mathbf{x}^{(j)}$:

$$\omega_{ij} = \mathbf{x}^{(i)\top} \mathbf{x}^{(j)}$$

- b) **Normalizamos para obtener α_{ij}** Esto consiste en normalizar las probabilidad para que

$$\sum_{j=1}^T \alpha_{ij} = 1$$

Por lo que se suele utilizar la función *softmax* para obtener este resultado así que:

$$\alpha_{ij} = \frac{\exp(\omega_{ij})}{\sum_{j=1}^T \exp(\omega_{ij})} = \text{softmax}([\omega_{ij}]_{j=1\dots T})$$

- c) **Computamos $\mathbf{z}^{(i)}$** como hemos expresado arriba.

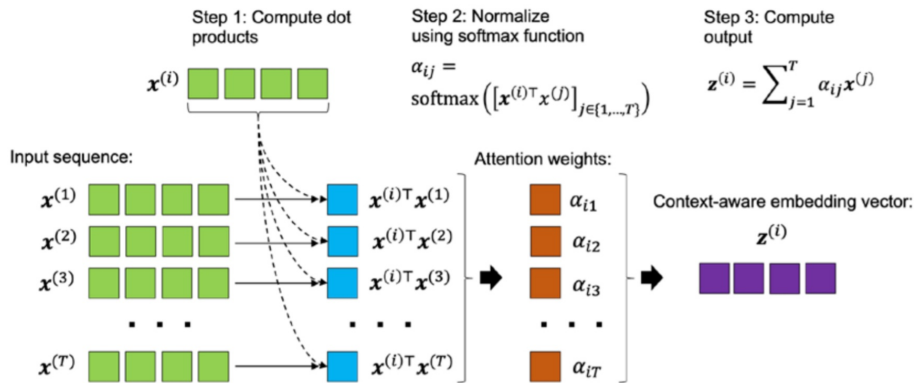


Figura 2.16: Proceso de *self-attention* con propósito ilustrativo. Image obtenida de [28].

Este proceso aplicado a cada una de las entradas se llama cabeza de atención, siendo las versiones en uso más complejas y utilizando diferentes matrices como *Query*, *Key* y *Value*, junto a diferentes estrategias de pesos. Ejemplos de estas utilizadas por las distintas arquitecturas estudiadas se repasan en el capítulo 4, pero una explicación más detallada queda fuera de los objetivos de explicación de este TFG.

Mencionar finalmente que además estas cabezas de atención suelen usar en en grupos, llamándose esto *multi-head attention* siendo esencialmente las mismas operaciones pero de forma matricial, un esquema de esto podemos verlo en la Figura 2.17, esto permite al modelo capturar diversas características y relaciones en los datos, enriqueciendo la representación aprendida. En el ViT, esta capacidad es crucial para comprender la estructura espacial de las imágenes.

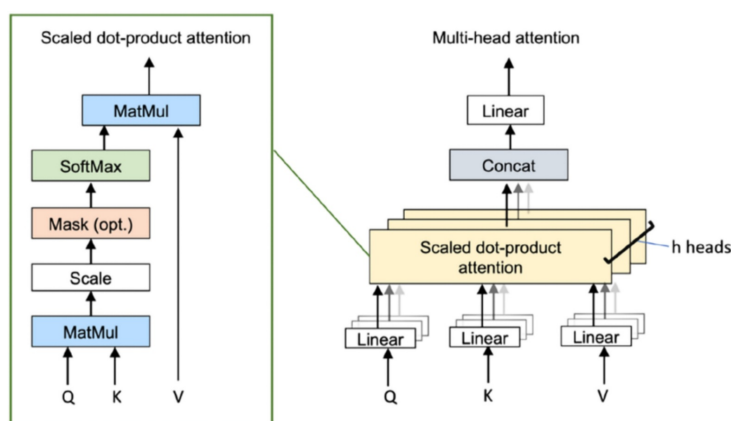


Figura 2.17: Diagrama de *Multi-head attention*. Image extraída de la publicación [28].

4. **ViT (*Vision Transformer*):** Esta familia de modelos de IA representan una adaptación del modelo *Transformer* original [45] al dominio de la imagen. Esta adaptación trata de aplicar una arquitectura con un resultado muy positivo en el procesamiento de texto, aplicándola a la imagen. En vez de utilizar capas convolucionales para extraer características locales, como en las CNN, ViT procesa la imagen como una secuencia de *tokens*, análoga a una secuencia de palabras en el contexto de procesamiento de lenguaje natural (NLP) y aplica la tecnología de la atención.

Esta arquitectura en su origen cuenta con dos bloques diferenciados y un paso previo de procesamiento de la entrada:

- a) **Preprocesado de la entrada.** Antes de poder introducir los datos en la red, debemos darles un sentido y posicionamiento para poder interpretar la imagen. Como

podemos ver en la Figura 2.18 Cada parche o fragmento de la imagen en la que se ha dividido previamente en el paso de tokenización se ha aplanado en un vector, representado en la figura como cada una de las entradas al modelo. Además, se le añade un *token* especial (CLS), utilizado para tareas de clasificación que, durante el entrenamiento va aprendiendo la importancia y características de la aplicación. A esto se le añaden "*codificaciones posicionales*", un vector añadido con información de la posición del parche en la imagen, ya que el modelo por sí solo no es sensible al orden de los elementos.

- b) **Encoder:** Esta secuencia de *tokens* una vez procesados se introduce en el *encoder*. Este consiste en bloques que alternan mecanismos de "*multi-head attention*", capas de MLP o "*feedforward*" una capa de normalización de los datos. Este mecanismo de "*multi-head attention*" es especialmente importante. Este permite al modelo aprender diversas relaciones espaciales entre los parches y regiones de interés en la imagen, y al ser varios procesos simultáneos, podemos intuitivamente comprenderlos como diferentes cabezas de atención procesando la misma secuencia desde múltiples perspectivas. Este proceso de *self-attention* permite que cada parche "preste atención" a todos los demás parches de la imagen, lo que favorece el modelado explícito de dependencias a lo largo de la red neuronal, a diferencia de las CNNs, donde la información global se acumula progresivamente mediante capas de convolución y *pooling*.

Al final de pasar por el *encoder* el CLS contiene una representación global de la imagen.

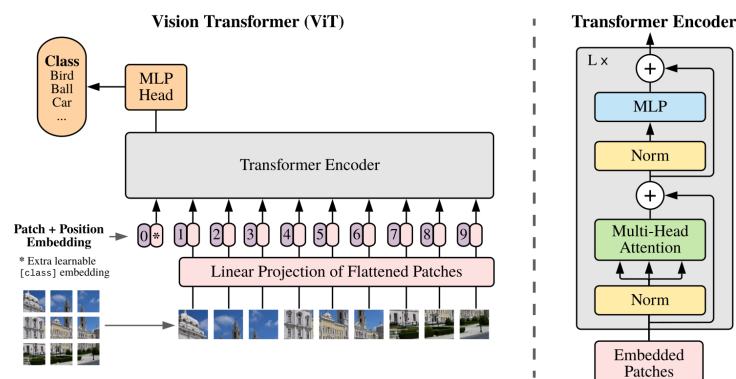


Figura 2.18: Esquema del *encoder* de una arquitectura *Transformer*. Imagen extraída de [46].

- c) **Decoder:** A partir de la representación de la entrada resultado del proceso dentro del *encoder*, esta información pasa al *decoder* a mitad de proceso. El *decoder* contiene tres subcomponentes Figura 2.19:

- Una capa de *self-attention* enmascarada que impide que el modelo vea *tokens* futuros durante el entrenamiento, asegurando una generación autoregresiva.
- una capa de atención cruzada permitiendo al *decoder* enfocarse en partes relevantes de la entrada codificada, combinando así información del contexto original con lo generado hasta el momento
- una red *feedforward* para combinar las distintas capas y su información en el mismo flujo además de aplicar transformaciones no lineales, ya que sus funciones de activación no son lineales, asegurando que el modelo aprende.

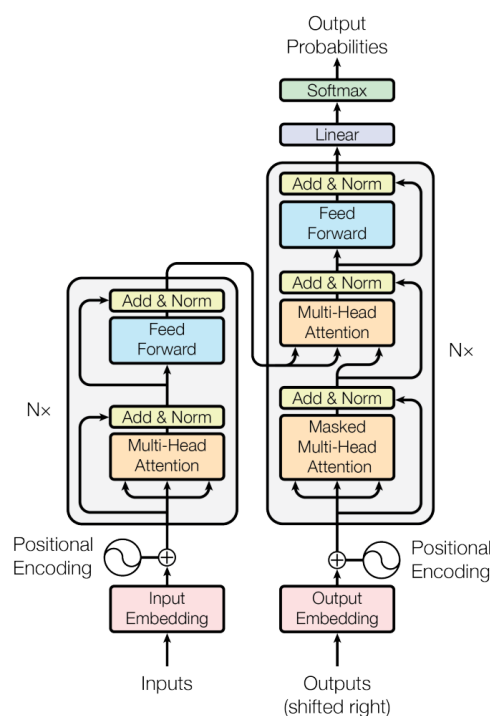


Figura 2.19: Esquema de la arquitectura original *encoder-decoder*. Imagen extraída de [46].

Al pasar por múltiples capas, el *decoder* va construyendo representaciones cada vez más ricas, que luego se transforman en *tokens* de salida, estos finalmente se convierten en un resultado mediante una capa de proyección final y una función *softmax*.

Mencionar que esta es la arquitectura original de las ViT, y si bien nos centraremos en esta arquitectura, hay modificaciones a esta, tales como arquitecturas solo *Encoder*, solo *Decoder* o aplicando distintas tecnologías y aproximaciones como *DEIT*, *YOLO*, ..., etc.

5. **Redes convolucionales (CNN):** Pese a no ser el sujeto de estudio de este TFG, debemos explicar esta arquitectura ya que será el punto de comparación de los resultados de las arquitecturas ViT, ya que la arquitectura más popular y extendida aplicada a problemas de visión.

Las CNN, son un tipo especializado de redes neuronales diseñadas para procesar datos con estructura en forma de matriz, como es el caso de las imágenes. A diferencia de las redes neuronales tradicionales como el caso de la MLP que conectan cada neurona con todas las neuronas de la siguiente capa, las CNN aprovechan la convolución.

La convolución es una operación matemática que aplica pequeños filtros que recorren la imagen para detectar características locales, como bordes, texturas o formas. Estos filtros permiten capturar patrones espaciales relevantes y podemos entenderlos como un "pre-procesado" de la imagen para poder obtener los datos más relevantes de manera eficiente.

El funcionamiento básico de una CNN consiste en aplicar sucesivas capas de convolución Figura 2.20, seguidas generalmente capas de neuronas artificiales y capas de *pooling*, este consiste en una reducción de la dimensión manteniendo la información más importante. Las capas convolucionales extraen características cada vez más complejas conforme se avanza en la red. Y finalmente, las representaciones aprendidas se pasan a una red neuronal convencional para tareas como clasificación o detección.

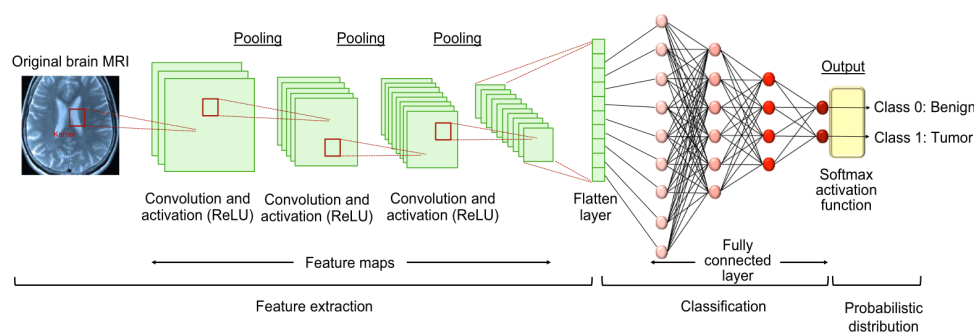


Figura 2.20: Esquema una arquitectura CNN aplicada a la neuroimagen. Imagen extraída de [47].

Mencionar que al igual que en el caso de las arquitecturas ViT, las CNN han evolucionado mucho y aplican distintas arquitecturas y tecnologías haciendo que cada modelo sea único, influenciando a las ViT en muchas arquitecturas híbridas como podemos ver en esta imagen Figura 2.21

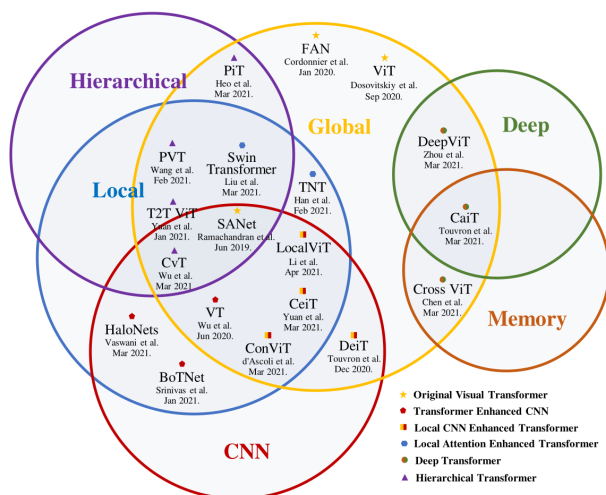


Figura 2.21: Esquema de las diferentes arquitecturas ViT. Imagen obtenida de [47].

2.3. Proceso de entrenamiento

El proceso de entrenamiento de una red neuronal constituye el corazón del aprendizaje profundo aplicado a la visión artificial. Durante este capítulo seguimos el proceso de este entrenamiento desde un punto de vista teórico a la vez que comprensible, de forma que facilite al lector comprender la implementación de este proceso en el capítulo de metodologías3.

En esta etapa, el modelo aprende a reconocer patrones visuales relevantes y desarrollar los mecanismos de atención, a partir del uso de las imágenes de los *datasets* 4. Para ello el proceso cuenta de los siguientes pasos previos al entrenamiento, que explicaremos siguiendo las recomendaciones del libro *Machine Learning with PyTorch and Scikit-Learn* [28]:

1. **Preprocesado:** Consiste en comprobar el estado de los datos y filtrarlos para su correcto funcionamiento, en el marco de este TFG, seleccionar imágenes de pólipos individuales para hacer una correcta detección y filtrar aquellas imágenes con pólipos distorsionados, es decir, cortados en el borde de la imagen, o imágenes sin pólipos.
2. **Selección de un algoritmo apropiado:** Como es el caso de estudio de este TFG, la arquitectura *Visual Transformer*.
3. **Selección de una *loss-function* y un optimizador adecuados:** En este caso CIoU y Adam.

Una vez tomadas estas decisiones, vamos a seguir los pasos dados en general para el aprendizaje supervisado explicando su aplicación a el marco de este TFG así como las distintas tecnologías escogidas para realizar estos pasos [28]:

1. **Forward propagation:** En la que las imágenes se introducen al modelo y este hace sus predicciones.
2. **Calculo del error.** Fase en la cual se mide la diferencia entre la salida predicha y el objetivo.
3. **Actualización de pesos** utilizando el error obtenido, actualizamos los pesos de la red neuronal para aprender de este.

2.3.1. *Forward-propagation*

La propagación hacia adelante o *back-propagation*, es el primer paso de la fase de entrenamiento, durante el cual una red neuronal transforma una imagen de entrada en una bbox predicha, recorriendo secuencialmente sus capas desde la entrada hasta la salida Figura 2.22. Este mecanismo es esencial tanto durante el entrenamiento como en la fase de inferencia, ya que determina cómo la red procesa y predice resultados a partir de los datos proporcionados.

Una definición más formal de esto es entender que la salida de una neurona de cualquier capa oculta h $\alpha_1^{(h)}$ = teniendo una entrada para esa neurona tal que $X = x_1^{(in)}, \dots, x_m^{(in)}$ y los pesos dados a esas entradas $W^{(h)} = w_1^{(h)}, \dots, w_m^{(h)}$ entonces definiendo la salida como [28]:

$$z_1^{(h)} = x_1^{(in)} w_1^{(h)} + x_2^{(in)} w_2^{(h)} + \dots + x_m^{(in)} w_m^{(h)} = W^{(h)\top} \cdot X$$

$$\alpha_1^{(h)} = \sigma(z_1^{(h)}) \text{ donde } \sigma(\cdot) \text{ es la función de activación}$$

Aplicando esto a cada neurona y pasando la información por todas las capas como podemos ver en la Figura 2.22 se obtiene la predicción final de la red tal que:

$$\mathbf{Z}^{(h)} = \mathbf{X}^{(in)} \mathbf{W}^{(h)\top} + \mathbf{b}^{(h)} \quad (\text{entrada de la capa oculta})$$

$$\mathbf{A}^{(h)} = \sigma(\mathbf{Z}^{(h)}) \quad (\text{salida de la capa oculta})$$

$$\mathbf{Z}^{(out)} = \mathbf{A}^{(h)} \mathbf{W}^{(out)\top} + \mathbf{b}^{(out)} \quad (\text{entrada de la capa de salida})$$

$$\mathbf{A}^{(out)} = \sigma(\mathbf{Z}^{(out)}) \quad (\text{salida de la capa de salida})$$

El resultado de este proceso depende de la estructura y objetivo de la red neuronal, en el marco de este TFG es una bbox con formato "*double corner*." (x1, y1, x2, y2). Siendo esta bbox la comparada con el objetivo en el siguiente paso.

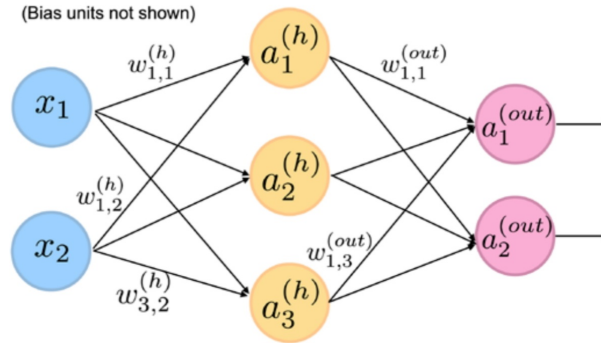


Figura 2.22: Esquema de una red MLP en el paso *forward-propagation*. Imagen obtenida de [28]

2.3.2. Cálculo de pérdida

Una vez obtenida la salida de la red se evalúa la diferencia entre esta y la salida esperada utilizando una función de pérdida o "*loss-function*" adecuada.

En el marco de este TFG como hemos explicado, usaremos por ser la que ha tenido mejor resultado experimental, tanto por su sentido teórico ya que tiene una relación con nuestra métrica de evaluación *IoU* y de las disponibles en este marco es la que tiene mejor resultado tanto teórico, véase Fig2.12, Haciendo uso de esta función explicaremos como se compara la salida obtenida durante la fase de *Forward-propagation* con las bbox objetivo y así obtener el error.

Complete Intersection over Union (CIoU) Esta elección podría haber sido al propia función de evaluación *IoU* ya que es lo que buscamos. El problema reside en que *IoU* es un valor visual y no representa fielmente el error entre los vectores de salida y de entrada, además que no tiene en cuenta otras métricas como la distancia de los centros de cada bbox. Para tener en cuenta el error entre la bbox de salida y la objetivo, podríamos haber usado una función que evalúe la distancia entre estos dos "vectores". Para ello, lo lógico es utilizar MAE o mejor aún MSE o L1.

Estas funciones de pérdida sin duda son útiles y funcionan, hemos descrito en 3. Pero finalmente la función más completa y descriptiva del error en el problema de detección que es este TFG es **CIoU**. Esto se debe a que combina óptimamente la métrica objetivo *IoU* con una comprensión de las coordenadas y la distancia euclídea entre los valores de salida y la salida

esperada tanto de sus centros como la coherencia de sus proporciones [41].

Una definición más formal de esta función de pérdida está compuesta y definida por:

1. ***IoU*** es la Intersección sobre la Unión entre las cajas predicha y real
2. ***d*** es la distancia euclidiana entre los centros de las cajas predicha y real.
3. ***v*** mide la coherencia de la relación de aspecto entre las cajas y se calcula como:

$$v = \frac{4}{\pi^2} \left(\arctan\left(\frac{w^{(obj)}}{h^{(obj)}}\right) - \arctan\left(\frac{w}{h}\right) \right)^2$$

Tal que $w^{(obj)}$ y $h^{(obj)}$ son la anchura y altura de la bbox objetivo y w y h la anchura y altura de la bbox predicha.

4. α es un parámetro de equilibrio positivo definido como:

$$\alpha = \frac{v}{(1 - \text{IoU}) + v}$$

Finalmente podemos definir la *loss function* CIoU $\mathcal{L}_{CIoU}$ como [41]:

$$\mathcal{L}_{CIoU} = 1 - \text{IoU} + \frac{d}{c} + \alpha v$$

La combinación de estas métricas en la misma función de pérdida es un gran avance y el cambio entre esta y "*smooth_l1_loss*" [48] ha sido de una mejora del *IoU* en *validation* del 8-15 % y hasta un 10-17 % en *test*.

2.3.3. Actualización de pesos

La actualización de pesos es la etapa clave del entrenamiento ya que es durante esta en la que se modifica la red neuronal y se calcula como debe afectar el error calculado en cada neurona. Esto permite que el modelo aprenda y mejore su rendimiento.

Para ello se utiliza una técnica llamada "*back-propagation*". Podemos entender el "*back-propagation*" de una forma intuitiva como la técnica para aprender los pesos en una red neuronal de forma muy eficiente, en la que el error en la **capa de salida** va remontando "hacia atrás" en la red por cada **capa oculta** hasta llegar a la **capa de entrada**.

Para hacer una definición más formal seguiremos la explicación dada por *Sebastian Raschka* en [28] y el esquema de la Figura 2.22.

Retomando la definición de las entradas y las salidas de la capa oculta h y la de salida out hecha en la *forward-propagation*, podemos entender el *back-propagation* como la aplicación de la regla de la cadena aplicada a cada uno de los pasos dados durante el *forward-propagation* como indican las **flechas en negrita** de la Figura 2.23. Haciendo ese proceso haciendo las derivadas parciales de los resultados de cada capa paso por paso tal que:

$$\frac{\partial L}{\partial w_{1,1}^{(out)}} = \frac{\partial L}{\partial a_1^{(out)}} \cdot \frac{\partial a_1^{(out)}}{\partial z_1^{(out)}} \cdot \frac{\partial z_1^{(out)}}{\partial w_{1,1}^{(out)}}$$

Esta derivada parcial es la responsable de actualizar los pesos de $w_{1,1}^{(out)}$. Estas fórmulas para su simplicidad y legibilidad omiten información como el bias o las funciones de activación, para seguir este proceso en mayor profundidad se puede seguir la explicación de *Sebastian Raschka* en [28].

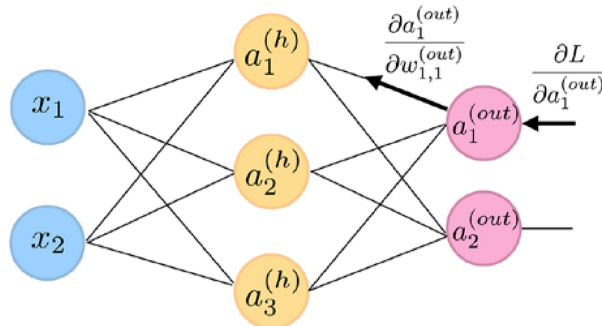


Figura 2.23: Esquema de una red MLP en el paso *back-propagation*. Imagen obtenida de [28].

2.4. El bucle de entrenamiento

El proceso descrito en la sección 2.3 es tan solo las bases de el paso de una sola imagen por el modelo. Durante el proceso real todas las imágenes de *train* pasarán por el modelo actualizando sus pesos. A esto lo llamamos una época, y durante el entrenamiento se realizan varias de ellas, en las que no solo se entrena el modelo, si no que se evalúa su rendimiento en esa época y se coteja el rendimiento con el conjunto de *validation*.

Durante el proceso de evaluación con el conjunto de *validation*, se pasan los datos por el modelo y se cotejan sus resultados pero sin realizar el paso de actualización de pesos. Final-

mente el modelo se evalúa de la misma manera con el conjunto de *test* y ese es el resultado que consideramos como precisión final del modelo.

Añadir que por motivos de eficiencia el paso de las imágenes no se realiza de uno en uno, si no que se realizan todas estas operaciones con varias imágenes de una forma matricial, para hacer el proceso más eficiente. A estos grupos de imágenes que se cargan juntas las llamamos *batches*, y suelen tener un tamaño de 5-10 imágenes por *batch*. Por este motivo se usan GPUs para el proceso de entrenamiento, ya que su fuerte es la computación en paralelo.

Todo este proceso naturalmente necesita de un software consistente y de un trabajo de adaptación y unificación de arquitecturas y métodos. Adaptación la cuál podemos leer en el siguiente capítulo de métodos 3

Capítulo 3

Métodos

En este capítulo, detallamos las metodologías empleadas y adaptación de diversas arquitecturas ViT y CNN para la de detección de pólipos en imágenes de colonoscopias. Una de las principales dificultades en el estudio de este TFG es unificar las salidas de cada modelo para que su única salida sea una bbox con formato *double corner* (x1, y1, x2, y2) que represente con precisión como cada modelo es capaz de detectar los pólipos.

Para lograr esta unificación, se ha hecho un análisis de cada arquitectura y sus salidas y utilizando diversas herramientas, incluyendo la incorporación de cabezas de detección o el filtrado de las salidas. En el caso de las cabezas de detección, estas facilitan una integración sencilla y eficiente en múltiples de las arquitecturas, mejorando la coherencia y precisión en la identificación de las regiones de interés.

Además de este análisis, se presentará una breve descripción del entorno de desarrollo esto incluye detalles sobre las bibliotecas y *frameworks* seleccionados, así como la configuración del entorno de trabajo, para proporcionar una visión completa de los recursos técnicos empleados en el proyecto y su replicación. También hacemos una demostración de las herramientas de software creadas para este TFG, con el fin de hacer comprensible su utilización o adaptación de este código para cualquier aplicación o uso futuro.

Por lo tanto, este capítulo proporciona una visión integral de las estrategias y aplicaciones desarrolladas, desde la selección y adaptación de modelos hasta la implementación técnica, estableciendo las bases para los resultados y conclusiones presentados en los capítulos posteriores.

3.1. Software y entorno de desarrollo

Para ofrecer la oportunidad de replicar este TFG y la reutilización del mismo para cualquier interesado, en esta sección explicamos brevemente el entorno utilizado y ejemplificamos el uso de las herramientas software creadas con el fin de hacer más comprensible el código para el lector y entender mejor la forma en la que hemos obtenido los resultados y datos de análisis presentados en capítulos posteriores de este TFG. El código desarrollado para este TFG y que explicamos brevemente durante esta subsección puede encontrarse en el repositorio de *GitHub* [49].

Entorno de desarrollo y librerías Python

Para la realización de este TFG hemos escogido el siguiente entorno:

- Editor de código: *Visual Studio Code 1.96.4*
- Sistema operativo: *Rocky Linux 8.8 (Green Obsidian)*
- GPU utilizada para entrenamiento: *NVIDIA GA102GL [A40]* [50]

Las librerías más importantes utilizadas son:

- python 3.12.8
- pytorch 2.5.1
- torchvision 0.20.1
- datasets 2.19.1
- scikit-learn 1.6.1
- numpy 1.26.4
- matplotlib 3.10.0
- seaborn 0.13.2

Software utilizado

Durante la realización de este TFG ha sido necesario desarrollar una serie de herramientas tanto para el procesamiento de los *datasets* como para el entrenamiento de los modelos una vez modificados, durante esta sub-sección explicaré y ejemplificaré el funcionamiento de las mismas para poder comprender mejor el proceso y el origen de los datos mostrados en capítulos posteriores.

Image Dataset Processor

El primer desafío a la hora de trabajar con imágenes de colonoscopias es la diversidad de los datos y fuentes, lo que naturalmente resulta en una diversidad tanto de formatos como organización y en general naturaleza de estas imágenes, lo que resulta en *dataset* que van desde conjuntos de datos organizados en *splits*, con dos tipos de resoluciones hasta metadatos clínicos, hasta *datasets* con imágenes sin clasificar con hasta +40 resoluciones distintas. Por lo que en este entorno tan variable es necesario unificar los datos en un mismo conjunto. Para procesar estos de forma eficiente, estandarizar su formato, analizar su composición y así poder utilizar más o menos datos según sea necesario haciendo una interfaz sencilla de utilizar, hemos desarrollado la clase *Image Dataset Processor* (IDP).

Esta clase procesa el *dataset* dado guardándolo como un diccionario de elementos. Ofrece herramientas y funciones para a partir de este diccionario salvar, modificar o obtener información relevante del mismo. Los detalles de su implementación no entran dentro del marco de estudio de este TFG, pero sí explicaremos un ejemplo de su uso para el caso de cargar el *dataset* de *Piccolo* [1]. Para cargar el *dataset* simplemente seguimos estos pasos:

1. **Inicializar la clase** con los parámetros que deseamos, nombre o resolución como vemos en la Figura 3.1. también puedes hacer uso de parámetros como: *json_path* para dar un nombre personalizado al JSON donde guardar los datos del *dataset* una vez procesado, *metadata_path* con metadatos clínicos, en el caso de piccolo para poder dar un análisis más exhaustivo de la composición del *dataset*.

```
# inicializamos el dataset con su nombre y formato de imagen
dt_Piccolo = idp(target_resolution=(400, 240), dataset_name="Piccolo")
```

Figura 3.1: Inicializar la clase IDP.

2. **Cargar las imágenes del *dataset***, para ello sencillamente debemos introducir los *paths* a

los directorios que contienen las imágenes como vemos en la Figura 3.2. Si no tenemos un archivo CSV con el tipo de luz de cada imagen y tenemos las imágenes distribuidas en directorios por tipo de luz, como es el caso del *dataset* de *PolypDB*, podemos usar el parámetro *dir_light_type* para indicar el tipo de luz de las imágenes guardadas. Toda la información dada no afecta a la creación de un conjunto de entrenamiento, pero si es ofrecida, se analizará.

```
# paths del dataset
paths_img = [
    # train paths
    (DS_PATH + "/piccolo_dataset/test/polyps",           # polyps
     DS_PATH + "/piccolo_dataset/test/masks",           # masks
     DS_PATH + "/piccolo_dataset/test/void",            # voids
     DS_PATH + "/piccolo_dataset/listado_WL-NBI_Piccolo.csv") # light type

    # cargamos los directorios con las imágenes en la clase
    for path in paths_img:
        dt_Piccolo.load_dataset(path[0], path[1], voids_path=path[2],
                                split=path[3], light_csv=path[4])
```

Figura 3.2: Cargamos las imágenes del conjunto de *train* del *dataset* de Piccolo en la clase IDP.

3. **Mostrar el análisis**, usando funciones como: *show_image* para mostrar imágenes del *dataset* cargado, *print_summary* para mostrar en forma de texto un resumen de la composición del *dataset* y *graph_summary* para mostrar los gráficos con esos mismos datos. Estas funciones son las que muestran los datos obtenidos y son la base del análisis de los *datasets* empleados descrito en el capítulo 4. Podemos ver una muestra de los datos obtenidos con *graph_summary* en la Figura 3.3

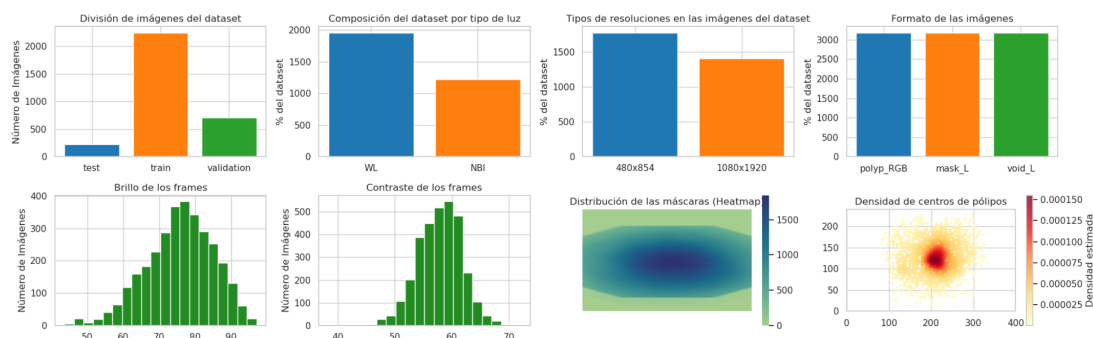


Figura 3.3: Análisis de la composición del *dataset* de Piccolo usando IDP.

4. **Obtener los *dataloaders***. Este es el último paso para preparar los datos para entrenamiento, en él, obtenemos los *dataloaders* [51]. Una estructura de *pytorch* que contiene los *splits* de cada modelo, y es requerida para cargar estos datos en entrenamiento de manera eficiente.

Para obtenerlos, la clase IDP implementa la función *get_dataloaders* como podemos ver en la Figura 3.4. Esta recibe parámetros como el tamaño del *batch* o si utilizar los *splits* definidos en el paso de *load_dataset* como vemos en la Figura 3.2. Otros parámetros útiles son: *analyze_splits* para dar información extra sobre los *splits* y su composición, *rand* utilizado para obtener aleatoriamente *x* cantidad de imágenes para cada *split* si no queremos entrenar con todos los datos cargados, para ello podemos usar los parámetros de entrada *train_split*, *val_split* y *test_split* para determinar el número de imágenes para cada *split*.

```
# Obtener los dataloaders de cada split
train_loader, val_loader, test_loader = dt_Piccolo.get_dataloaders(
    BATCH_SIZE,
    use_premade_splits=True)
```

Figura 3.4: División del *dataset* en Dataloaders usando la clase IDP.

Train Model

Esta segunda clase es la encargada del entrenamiento de todos los modelos así de la obtención de valiosa información del rendimiento del modelo durante el entrenamiento y otros detalles que puede ser muy útil a la hora de analizar el modelo y corregir tendencias negativas.

Al igual que la clase IDP esta clase trata de resolver el problema de unificación, siendo suficientemente generalista y ofreciendo una estructura capaz de entrenar cualquier modelo debidamente modificado. Esta clase sencillamente utiliza unos *dataloaders*, que es la estructura utilizada por *pytorch* para introducir datos a un modelo, y entrena el modelo ofreciendo funcionalidades para dar datos de análisis.

El uso de esta clase empieza por dar una serie de pasos previos definiendo las herramientas usadas para entrenar:

1. **Obtener los *dataloaders*** usando la función de IDP
2. **Definir la *loss function***. Como hemos descrito en capítulos anteriores, usaremos la CIoU. Esta ya está implementada de forma óptima y eficiente en la librería *torchvision* [52], Y la definimos como podemos ver en la Figura 3.5.
3. **Definir el optimizador**. Al igual que la *loss function* el optimizador utilizado elegido en capítulos anteriores es Adam, y tiene una implementación optimizada en la librería *torch* [53], y podemos observar su implementación en la Figura 3.5. Mencionar que la

tasa de aprendizaje de $lr = 5 \cdot 10^{-5}$ es experimentalmente adecuada y con ella hemos obtenido resultados adecuados.

```
# definimos la loss function 
def loss_fn(target, pred):
    return complete_box_iou_loss(pred, target, reduction='mean')

# el optimizador Adam 
optimizer = Adam(model.parameters(), lr=5e-5)
```

Figura 3.5: Definición de la *loss function* y el optimizador.

Una vez dados estos pasos previos definiendo las herramientas utilizadas en el entrenamiento, podemos entrenar el modelo simplemente en dos pasos:

- **Inicializando la clase *Train Model*.** Simplemente indicando: el modelo y incluyendo la loss function y optimizador definidos en la Figura 3.5, como podemos ver en la Figura 3.6. Otros parámetros útiles pueden ser: *eval_pred* para indicar que quieres evaluar la predicción y ofrece más datos, como un *heatmap* con los centros de las bbox objetivo y predichas, para evaluar el modelo con más datos de entrenamiento, otro parámetro es *meta_path* para utilizar los metadatos clínicos del *dataset* de *Piccolo* [1].
- **Entrenamiento del modelo.** Para ello simplemente llamamos a la función *train_model* y incluimos parámetros como: el modelo, el número de épocas y los dataloaders obtenidos anteriormente. Esta implementación podemos verla en la Figura 3.6.

```
# Inicializamos la clase 
trainer = trm(model, loss_fn=loss_fn, optim=optimizer)

# Entrenamos el modelo 
train_results = trainer.train_model(NUM_EPOCH, TRAIN_RES,
                                     train_loader, val_loader, test_loader)
```

Figura 3.6: Uso de la clase *TrainModel* para entrenar un modelo.

- **Analizar los datos de entrenamiento.** Finalmente la clase *TrainModel* al igual que *IDP*, implementa una función para obtener los resultados de entrenamiento como es *show_results*.

Finalmente añadir el detalle de que estas funciones están hechas para poder trabajar guardando sus análisis y resultados en un JSON y cargándolos de nuevo para agilizar el proceso de desarrollo, para más detalles de esta implementación, consultar el *GitHub* del TFG [49].

3.2. Implementación de modelos

En el entorno del desarrollo de IA las arquitecturas de uso público son tan variadas y con tal diversidad de fuentes, que es natural encontrarse arquitecturas adaptadas más o menos, o con más o menos componentes. Por este motivo en esta sección explicamos la naturaleza y origen de los modelos utilizados, además de explicar cómo y con qué herramientas hemos adaptado su arquitectura a el problema de detección que aborda este TFG.

3.2.1. YolosBBoxOnly

Esta arquitectura pertenece a la familia de modelos YOLO cuya versión original YOLO-base (You Only Look Once) fue introducida en 2015 por Joseph Redmon [54]. Esta familia de modelos, desde su modelo original, en lugar de seguir un enfoque tradicional, que entonces eran las arquitecturas CNN, el autor propone una arquitectura basada en los siguientes conceptos:

- **Dividir la imagen en una cuadrícula**
- **Predecir para cada cuadrícula**, en este caso predice una bbox y una clase, originariamente las etiquetas de clases del *dataset* de COCO [55].

Con estos dos sencillos pasos el modelo es capaz de procesar imágenes de una sola pasada haciendo el modelo más eficiente. Pero esta familia de modelos utiliza diferentes aproximaciones para cada versión, esta es *YOLOsBBoxOnly* propuesta por Yuxin Fang y colaboradores en "*You Only Look at One Sequence: Rethinking Transformer in Vision through Object Detection*" [56], una arquitectura que integra una ViT pura como una *backbone*, la arquitectura cuenta con los siguientes componentes clave:

- **Backbone ViT (YolosModel)**: Como hemos mencionado, este consiste en un ViT que divide la imagen en parches, los cuales se linealizan y se procesan a través de capas de atención para extraer características globales.
- **Atención escalada por producto punto (SDPA)**: Esta versión avanzada de las cabezas de atención, utiliza la implementación nativa de *PyTorch* para la atención, optimizando la eficiencia computacional durante la inferencia.
- **Normalización de capa (LayerNorm)**: Se aplica antes y después de las capas de atención para estabilizar el entrenamiento y mejorar la convergencia.

- **Cabezas de predicción o *YolosMLPPredictionHead***: Dos cabezas de predicción totalmente conectadas: una para la clasificación de etiquetas de clase y otra para la regresión de las coordenadas de las cajas delimitadoras.

Estas tecnologías las combina en un *encoder* ViT cuya entrada será de una resolución compatible con los *patches* creados por la arquitectura de 16x16. La arquitectura resultante podemos observarla en la Figura 3.7:

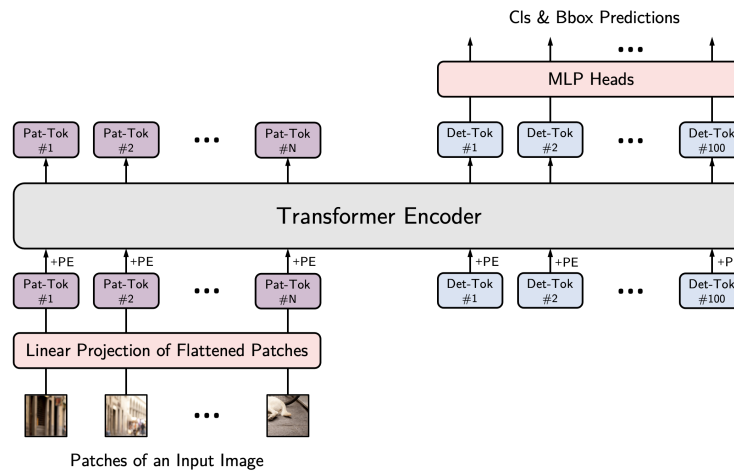


Figura 3.7: Arquitectura del modelo ViT. Imagen obtenida de [56].

Modificaciones de YolosBBoxOnly

Este modelo, que hemos obtenido de *hugging face* [57], está preparado para tanto la detección como para la clasificación ya que está pre-entrenado para la detección y clasificación en los datos del *dataset* de COCO [55], por lo que las modificaciones para este serán sencillas. Para ello solo debemos modificar dos elementos:

- **Entrada:** La entrada debe ser de una imagen compatible con los *patches* del modelo de 16x16, para ello, sencillamente elegimos una resolución de (480, 240) para respetar en la medida de lo posible las proporciones de las imágenes, y reducir la imagen lo suficiente para ser eficientes en entrenamiento.
- **Salida:** En este caso es un poco más complejo. Primero debemos congelar la cabeza de clasificación ya que no la vamos a usar, así no se actualizan los pesos de la misma ni corrompemos capas anteriores con sus valores.

La dificultad reside en obtener la bbox mejor predicha, ya que solo queremos una bbox como salida del modelo, pero el modelo YOLO obtiene una predicción por *patch* que combina (probabilidad de la clase, bbox asociada).

Para solucionar y simplificar esta salida, solamente debemos obtener la clase con más probabilidad, ya que esta tiene asociada la bbox con más confianza del modelo.

Con estas modificaciones, podemos introducir el *dataset* obtenido con IDP y hacer un *fine-tuning* del modelo en nuestros datos de CCR obteniendo sus resultados. Para más detalles de la implementación, consultar el *GitHub* del TFG [49]

3.2.2. YOLOv8

Esta arquitectura pertenece también a la familia de YOLO [54], y al igual que el modelo de *YolosBBoxOnly*, este modelo hace *patches* de 16x16 píxeles a partir de la imagen de entrada, además al igual que el modelo explicado, este predice por cada uno de estos *patches* una tupla compuesta por (clase de COCO, bbox asociada).

Por este motivo, tomaremos las mismas medidas para modificar el modelo *YOLOv8* y prepararlo para solo la detección de bbox.

Pero este modelo no es un ViT, *YOLOv8* tiene una arquitectura radicalmente distinta a *YolosBBoxOnly*, ya que esta es una CNN, y usaremos este modelo para comparar su resultado con las arquitecturas ViT, formando parte del grupo de control de este TFG.

La arquitectura de *YOLOv8* es una CNN y como explicamos en capítulos anteriores esta es una arquitectura basada en la aplicación de "filtros de imagen" encargados de ofrecer a la red neuronal final una imagen modificada para resaltar los elementos importantes para la detección y clasificación. Como cabe esperar esto es más complejo en la realidad del modelo *YOLOv8* y combina diferentes tecnologías para dar ese resultado, resaltar los siguientes bloques:

- **C2f** o "*Cross-Stage Partial with Fusion*": Este bloque es usado a lo largo de la red, consiste en dividir la entrada, para luego procesar una parte mediante *Bottlenecks* y luego fusiona las salidas junto con la entrada original para mejorar la eficiencia y la reutilización de características.
- **Convolutacional**: estos son los bloques base de la red, consisten en aplicar un filtro a la imagen o "*convolución*", gracias a esto, se extraen las características. Luego se normaliza

Recognition at Scale" [46]. Ambas utilizan las siguientes herramientas en su arquitectura:

- **Patch Embedding:** La imagen de entrada se divide en parches de tamaño fijo de 16x16. Cada parche se aplanan y se proyecta linealmente a un espacio de características de dimensión 384 para *ViT-Small* y 1024 para *ViT-Large* mediante una convolución 2D.
- **Positional Encoding:** agregan codificaciones posicionales a los *embeddings* de los parches para conservar la información de la posición espacial.
- **Bloques del *encoder*:** Estos forman parte de la composición de la red neuronal y se componen de:
 1. **LayerNorm:** Este paso que normaliza la entrada, cosa que estabiliza y acelera el entrenamiento.
 2. **Multi-Head Self-Attention:** Mecanismo de atención que combina varias cabezas de atención.
 3. **MLP:** Una red neuronal utilizada para introducir no linealidad al bloque y capacidad de aprendizaje.
 4. **Dropout:** Este paso consiste en ignorar algunas salidas de forma aleatoria, esto evita el sobreajuste.

Combinando estas herramientas se compone un bloque del *encoder* como vemos en la Figura 3.9.

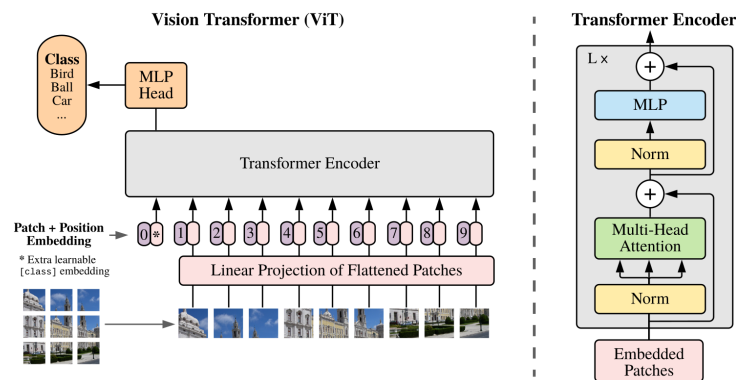


Figura 3.9: Esquema del *encoder* de una arquitectura *Transformer*. Imagen obtenida de citeimageWorth16x16.

Evaluamos estas arquitecturas para ver la eficiencia del modelo ViT y su capacidad de generalización, ya que la diferencia entre ambas reside en su tamaño. *ViT-Small* utiliza capas con un

tamaño de características de 384 y su *encoder* se compone de 12 bloques de *encoder*. En cambio *ViT-Large* utiliza el doble de bloques, 24, y estos trabajan en un espacio de características mayor de 1024.

Modificación del modelo

Ambos modelos obtenidos de la librería *timm* [60] ofrecen una arquitectura *encoder* como hemos explicado, por lo que en ambos casos debemos añadir un método para procesar la salida del *encoder*. Para ello, hemos escogido una **cabeza de detección**, como podemos ver en la Figura 3.10 esta consta de:

1. **Capa de entrada:** Esta recoge la salida del *encoder*, 384 para *ViT-Small* y 1024 para *ViT-Large*, y la pasa por una capa de neuronas cuya función de activación es "*ReLU*" compuesta por 384 neuronas.
2. **Capa oculta:** Capa de neuronas añadida para dar mayor complejidad y capacidad de aprendizaje al modelo, su entrada es el resultado de la **capa de entrada** y está compuesta igualmente por 384 neuronas cuya función de activación es "*ReLU*".
3. **Capa de salida:** Finalmente esta capa procesa la salida de la **capa oculta** con 4 neuronas sin función de activación, lo que hace que su salida sea lineal y por lo tanto nos de el resultado objetivo que es una bbox con formato (centro bbox eje x, centro bbox eje y, ancho, alto)

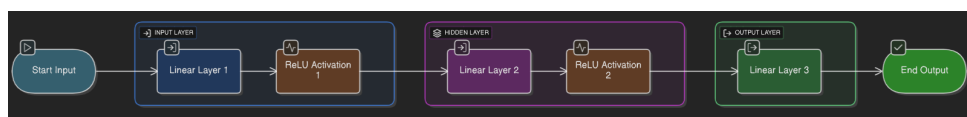


Figura 3.10: Esquema de la composición de la cabeza de detección.

El objetivo de incluir estas arquitecturas es poder comprobar cómo se comporta las arquitecturas ViT puras en el contexto de la detección de pólipos en imágenes de colonoscopias. Ya que son un modelo simple y complejo de ViT, para más detalles de su implementación consultar el GitHub de la asignatura [49].

3.2.4. DenseNet121

DenseNet121 es una variante de la familia *DenseNet* presentada por Gao Huang y colaboradores en *Densely Connected Convolutional Networks* [61]. Esta arquitectura ha sido seleccio-

nada para dar más diversidad al conjunto de control y debido a ser la mejor arquitectura CNN en imagen médica en el estudio de Satoshi Takahashi y colaboradores *Comparison of Vision Transformers and Convolutional Neural Networks in Medical Image Analysis: A Systematic Review* [11].

Esta familia de arquitecturas se basa en la tesis propuesta por Gao Huang que las CNN pueden ser mucho más eficientes y precisas si además de ser más profundas, cada capa tiene conexiones más cortas respecto a la entrada y la salida. Por ende esta arquitectura introduce las conexiones directas densas entre cada capa, haciendo que cada capa reciba como entrada los mapas de características de *todas* las capas anteriores y pasa los suyos a *todas* las posteriores.

DenseNet121 se caracteriza por crear un grafo con $\frac{L(L+1)}{2}$ conexiones para las L capas, de forma que cada capa está conectada de forma *concatenada* a todas las anteriores como podemos ver en la Figura 3.11 y no de forma secuencial como otras arquitecturas CNN como *ResNet*, haciendo que el flujo de información entre capas sea más eficiente y el número de parámetros se reduzca significativamente.

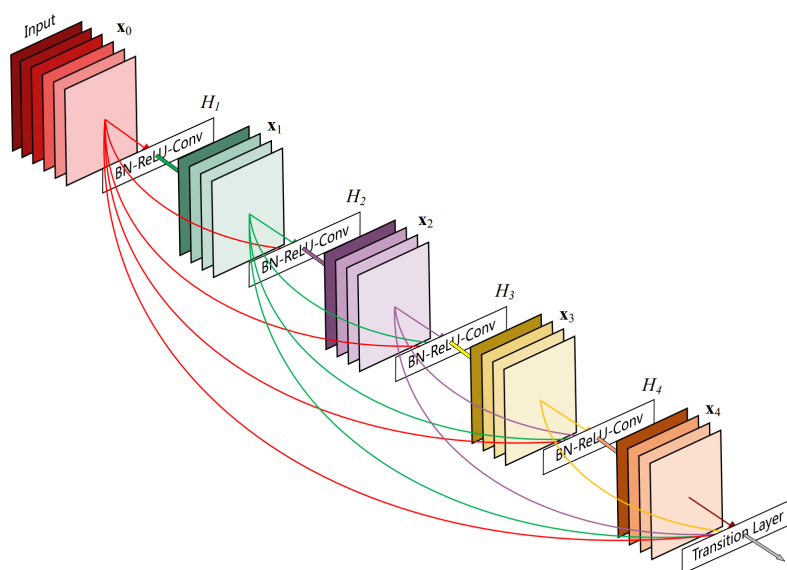


Figura 3.11: Esquema de la arquitectura y conexión entre capas de *DenseNet*. Imagen obtenida de [61].

Modificaciones de DenseNet

Este modelo al ser una CNN bastante compleja requiere de un entendimiento de su capa de salida y entrada para entender cómo modificarla:

- **Entrada:** Este modelo al ser una CNN su entrada es flexible ya que se le aplica un re-

escalado en las primeras capas y finalmente se convierte en un mapa de características, por lo que usamos la resolución 384x384 para mantener la comparabilidad con el resto de arquitecturas

- **Salida:** Las CNN trabajan internamente con mapas de características, lo que hace que obtener una salida coherente sea delicado.

Para obtener una salida coherente, el modelo ha sido importado sin capa de clasificación y por lo tanto su salida se pasa por una capa de *Global Pooling* que transforma el último mapa de características (*tamaño batch*, 1024, *Altura mapa*, *Anchura mapa*) a un tensor de características (*tamaño batch*, 1024)

Una vez entendida la capa de salida de la red, añadimos a esta una cabeza de detección similar a la utilizada en las arquitecturas ViT, haciendo al modelo capaz de realizar la tarea de detección requerida para este TFG, más detalles de su implementación el GitHub del TFG [49].

3.2.5. Data-efficient Image Transformer (DEiT)

La arquitectura DeiT es una variante optimizada de la arquitectura ViT presentada por Hugo Touvron y colaboradores en *Training data-efficient image transformers & distillation through attention* [62]. Este modelo mejora la eficiencia en el uso de datos y recursos computacionales introduciendo un mecanismo de *destilación* basado en atención. Este mecanismo consiste en un "token" que permite al modelo aprender de un "profesor" (una red neuronal ya entrenada que generalmente es una CNN) durante el entrenamiento, mejorando así su rendimiento sin necesidad de grandes conjuntos de datos adicionales.

Este *token* es la clave de la arquitectura que podemos ver en la Figura 3.12, ya que, y parafraseo la explicación de Hugo Touvron en la página 7 de "*Training data-efficient image transformers & distillation through attention*" [62], **El token de destilación interactúa con el token de clase y los patches en las capas de "self-attention"**, estos sirven para influir la entrada con las predicciones de una red neuronal "profesor" su peso es aprendido durante el entrenamiento.

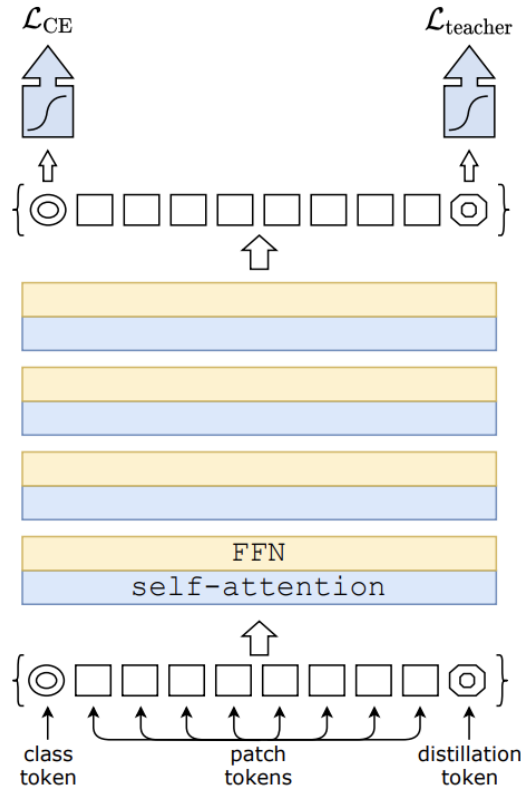


Figura 3.12: Esquema de la arquitectura DeiT. Imagen obtenida de [62].

El resto de la arquitectura es idéntica a la arquitectura ViT, con un espacio de características de 768 y 12 bloques de *encoder*.

Modificaciones del modelo

Al igual que el caso de las arquitecturas ViT, el modelo presenta solo el *encoder*, por lo que le añadimos igualmente una cabeza de detección idéntica a las aplicadas en los modelos ViT. El modelo ha sido obtenido de la librería *timm* [60]. Para más detalles en la implementación del modelo, consultar el GitHub de la asignatura [49].

3.2.6. Pyramid Vision Transformer (PVT)

Esta es otra arquitectura basada en la ViT, propuesta por Wenhai Wang y colaboradores en la publicación *Pyramid Vision Transformer: A Versatile Backbone for Dense Prediction without Convolutions* [63]. Esta arquitectura está diseñada para procesar imágenes de entrada de forma

que va reduciendo capa a capa su dimensión, esto significa intuitivamente, que concentra la información en capas más pequeñas para mejorar la eficiencia al final del modelo, pero con unas capas iniciales grandes para poder entender adecuadamente la imagen.

El modelo concreto de esta arquitectura que usamos es el "*PVTv2_b3*" que constará de tres bloques por etapa. Vamos a repasar los elementos claves de esta arquitectura, se pueden ver en la Figura 3.14. Los elementos que componen la arquitectura son:

- **Estructura en 4 etapas:** Esta es la estructura general del modelo, ya que el *encoder* de esta ViT a diferencia de las versiones base como Vit small y large, esta consta de 4 etapas cuyo tamaño va menguando, dando nombre a la arquitectura de pirámide. Cada una de estas etapas en el caso del modelo estudiado se compone de tres bloques idénticos.
- **bloque de *encoder*:** En este caso su composición es prácticamente idéntica al bloque *encoder* de las arquitecturas ViT, pero su tamaño de características va menguando.

La principal diferencia es la cabeza de atención "*spatial-reduction attention*" (SRA). La **SRA** es una variante de las cabezas de atención, que incluye un paso previo de concatenación de las entradas K y V como podemos ver en la Figura 3.13, lo que reduce la dimensionalidad de la entrada optimizando y reduciendo el costo computacional.

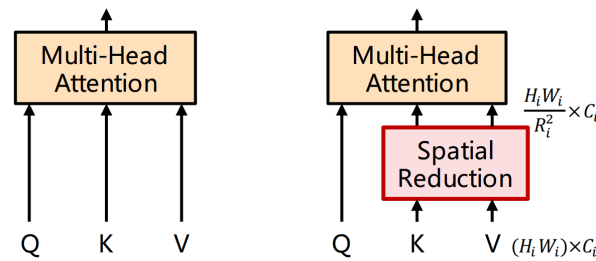


Figura 3.13: Cabeza de atención estándar (izquierda), SRA (derecha). Imagen extraída de [63].

Esta arquitectura forma parte de las arquitecturas estudiadas en este TFG al ser una modificación completamente basada en la arquitectura ViT. Esta arquitectura ha sido obtenida de la librería *timm* [60], para más detalles de su implementación, consultar el *gitHub* del TFG [49].

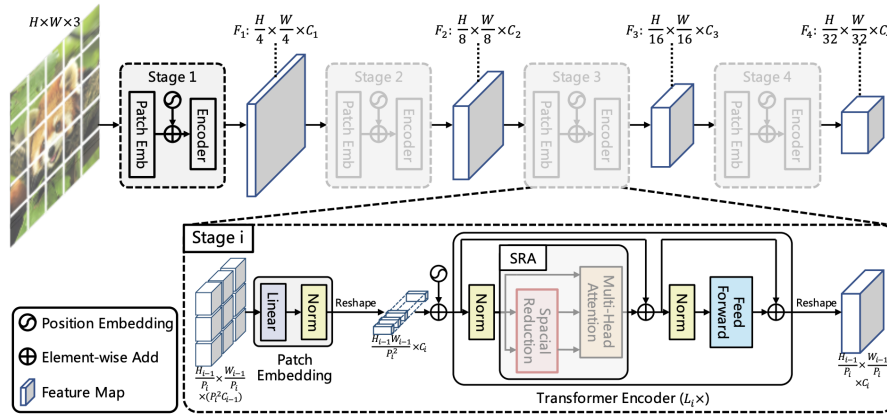


Figura 3.14: Esquema de la arquitectura PVT. Imagen extraída de [63].

3.3. Síntesis de los modelos presentados

En este capítulo se han descrito en detalle las arquitecturas de redes utilizadas para abordar el problema de detección de este TFG.

Estas arquitecturas han sido seleccionadas para representar una muestra del potencial del modelo ViT como son *ViT-small* y *ViT-large*, además de las distintas tecnologías y modificaciones de los modelos *transformer* como son *DeiT* y *PVT*. Incluimos la arquitectura *YOLOv8* para comparar los resultados de los modelos anteriores con el de modelos más tradicionales en la visión artificial como las CNN.

El código desarrollado durante el trabajo de este TFG permite no solo replicar los resultados presentados en este, si no que puede ser usado para el entrenamiento de cualquier modelo en el mismo contexto de este TFG o utilizarlo para crear soluciones propias para otros problemas o aplicaciones.

Finalmente mencionar que esta es la base técnica y práctica en la que se enmarcan los próximos capítulos, donde se abordarán el estudio de los datos utilizados, los experimentos realizados y se discutirán los resultados obtenidos, analizando el rendimiento de cada arquitectura bajo distintas configuraciones y conjuntos de datos.

Capítulo 4

Base de datos y análisis descriptivo

Este capítulo tiene como finalidad presentar las bases de datos utilizadas en el TFG. Estos y su composición son de vital importancia ya que definen la información que tiene la red neuronal del problema, por lo tanto deben ser debidamente seleccionadas para representar adecuadamente el problema y además dar la capacidad a los modelos de generalizar y aprender debidamente. El capítulo incluye el análisis exploratorio de datos de cada una de los datasets, una descripción de las etapas de preprocesamiento y finalmente una descripción y composición de los conjuntos de datos usados para el entrenamiento de los modelos.

4.1. Piccolo

El *dataset Piccolo* es el *dataset* base utilizado para el estudio de los modelos. Este *dataset*, de acceso bajo solicitud, es especialmente interesante, y lo utilizamos como base de los datos usados, ya que presenta no solo imágenes de pólipos variadas con sus máscaras, si no además una división de las imágenes por lesión y una serie de datos metaclínicos que son muy útiles para balancear y entender la diversidad de los datos. Añadido a esto, hemos utilizado estos datos en particular ya que durante la realización de este TFG hemos colaborado con el Centro de Cirugía de Mínima Invasión Jesús Usón (CCMIJU) y los investigadores que han creado este *dataset* Luisa F. Sánchez-Peralta y J. Blas Pagador, brindando un apoyo y ayuda inestimable para la correcta interpretación y aplicación de los datos.

Este *dataset* incluye 3433 imágenes de pólipos variadas en dos tipos de luz "*white light*" (WL) y "*Narrow-Band Image*" (NBI), provenientes de 76 lesiones en 40 pacientes distintos, como indica Luisa F. Sánchez-Peralta, J. Blas Pagador y colaboradores en *PICCOLO White-Light and*

Narrow-Band Imaging Colonoscopic Dataset: A Performance Comparative of Models and Datasets [13]. Esta información es presentada en unos conjuntos (*train*, *test*, *validation*) ya hechos por los creadores, que garantizan la independencia de los datos para entrenamiento y evaluación de los modelos.

Preprocesado de *Piccolo* y análisis

El *dataset* de *Piccolo* presenta algunos inconvenientes en sus *splits* originales como son:

- **Máscaras con ruido:** Estas presentan píxeles sueltos en la imagen que distorsionan el tamaño de las bbox objetivo creadas a partir de la máscara.
- **Imágenes con varios pólipos:** Para la aplicación de este TFG y el correcto entrenamiento, las imágenes de entrenamiento se han limitado a imágenes con un solo pólipo, ya que las imágenes con múltiples pólipos son minoritarias y las cabezas de detección no están preparadas para la detección múltiple. Además este no es necesario en la aplicación real ya que marcaría distintos pólipos según aparecen distintos fotogramas.
- **Imágenes vacías:** En el contexto de detección de este TFG utilizamos exclusivamente imágenes con pólipos. Estas imágenes suponen ruido a filtrar del *dataset* original, ya que representan una cantidad muy pequeña de las imágenes (19/2203 en *train* (0,86 %) y 24/897 imágenes en *validation* (2.68 %)), y por lo tanto mejoramos la estabilidad del entrenamiento al eliminarlas.
- **Pólipos diminutos:** Hemos filtrado aquellas imágenes donde el pólipo representaba menos de un 0.5 % de la imagen y por lo tanto distorsionaba el entrenamiento.

Una vez filtradas y redistribuidas estas imágenes el *dataset*, los resultados de entrenamiento tienen menos ruido y por lo tanto ayudamos al modelo a tener curvas de aprendizaje más estables como podemos ver en la Figura 4.2, especialmente en *validation*.

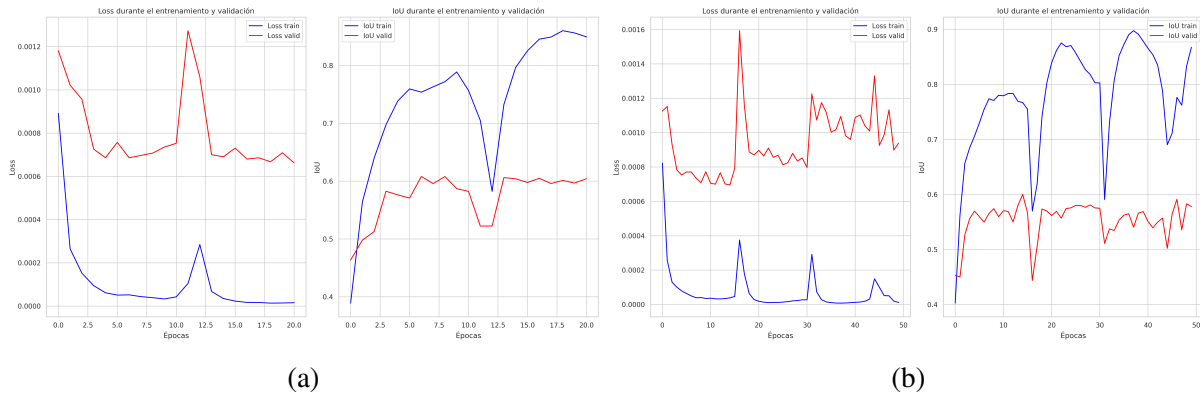


Figura 4.1: Comparación de rendimiento del modelo *ViT-Large* antes y después del pre-procesado. (a) Rendimiento del modelo *ViT-Large* tras el preprocesado; (b) Rendimiento del modelo *ViT-Large* antes de aplicar el preprocesado.

El *dataset* después del filtrado se compone de 3177 con la composición que podemos ver en la Tabla 4.1 y más visualmente en Figura 4.2:

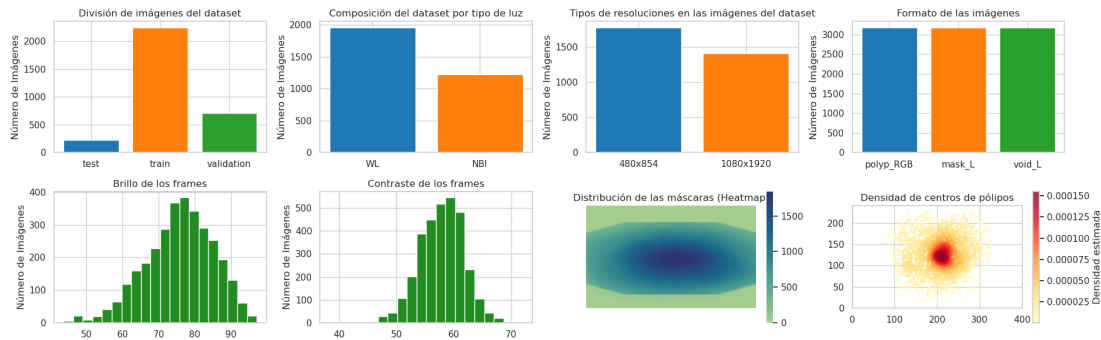


Figura 4.2: Gráficos de la composición del *dataset Piccolo*. De izquierda a derecha y de arriba a abajo: Número de imágenes por *split*, Número de imágenes por tipo de luz, Número de imágenes por resolución, Número de imágenes por formato, Número de imágenes por brillo, Número de imágenes por contraste, *Heatmap* compuesto de la disposición de las máscaras en la imagen (re-escaladas a la misma resolución), distribución de los centros de los pólipos en la imagen (re-escalado a la misma resolución).

Tabla 4.1: Resumen de la composición del *dataset Piccolo*.

Ítem	Valor
Total de imágenes	3177
Resoluciones	
854x480	1771
1920x1080	1406
Tipos de luz	
WL	1954
NBI	1223
Splits	
<i>Train</i>	2247
<i>Validation</i>	707
<i>Test</i>	223
Canales por imagen	
Imágenes de pólipo	3177
Máscaras binarias	3177
Filtro Void	3177
Volumen medio pólipo / imagen	21.29 %
Volumen medio bbox / imagen	61.60 %
Distancia media centro pólipo / centro imagen	51.45px

Como podemos ver el *dataset* de Piccolo es adecuado para el entrenamiento además que hace especial énfasis en la distribución de los pólipos de forma que se repartan por la imagen y no se concentren en el centro, como vemos en el *heatmap* de la Figura 4.2.

También mostrar la cantidad de imágenes que existen de cada tipo de lesión según la clasificación de París como vemos en la Figura 4.3 y la Tabla 4.2.

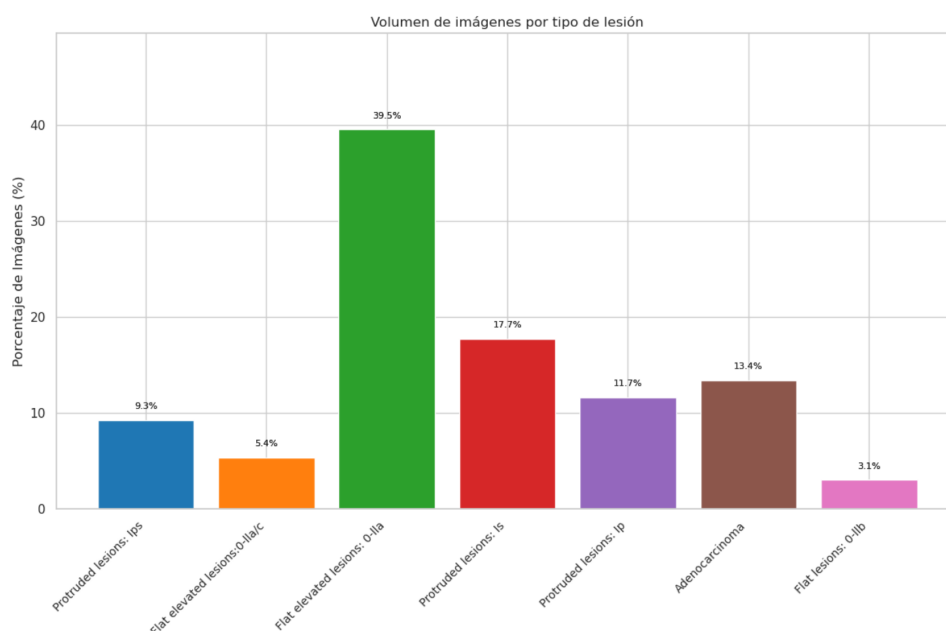


Figura 4.3: Distribución de imágenes por tipo de lesión en el *dataset* de *Piccolo*.

Tabla 4.2: Resumen datos clínicos de *Piccolo*.

Tipo de lesión	Número de imágenes
Lesiones pediculadas Ips	294
Lesiones planas elevadas O-Ila/c	170
Lesiones planas elevadas O-Ila	1256
Lesiones sésiles Is	563
Lesiones pediculadas Ip	371
Adenocarcinoma	426
Lesiones planas O-Ilb	97

4.2. PolypDB

PolypDB es un *dataset* presentado por Dabesh Jha y colaboradores en la publicación *PolypDB: A Curated Multi-Center Dataset for Development of AI Algorithms in Colonoscopy* [64]. Este *dataset* de acceso público recoge imágenes de centros médicos en Noruega, Suecia y Vietnam dando variedad a las imágenes y herramientas utilizadas. Le objetivo de la recopilación de estos datos es dar una muestra más variada y fehaciente de las imágenes que una aplicación real puede encontrarse, incluyendo imágenes obtenidas de máquinas de colonoscopias de distintas marcas y con 5 tipos distintos de luz.

Este conjunto de datos incluye 3934 imágenes, 3588 (91.20%) con luz WL y los otros tipos de luz, están infra-representados con un reparto tal que: *Blue Light Imaging* (**BLI**) **70 imágenes** (1.78%), *Flexible Imaging Color Enhancement* (**FICE**) **70 imágenes** (1.78%), *Linked Color Imaging* (**LCI**) **60 imágenes** (1.53%) y finalmente **NBI** con **146** (3.71%). La composición y análisis del *dataset*

Preprocesado de *PolypDB* y análisis

Una vez visto los datos básicos del *dataset*, pasamos al preprocesado y análisis del mismo. Este *dataset* será utilizado íntegramente para el *split* de entrenamiento, dando mayor variedad a los datos y garantizando la independencia entre los *splits*. Los datos de este *dataset* pasarán por los mismos pasos de preprocesado dados con el *dataset* de Piccolo. La composición y características del *dataset* resultante podemos observarlo en la Figura 4.4 y la Tabla Tabla 4.3

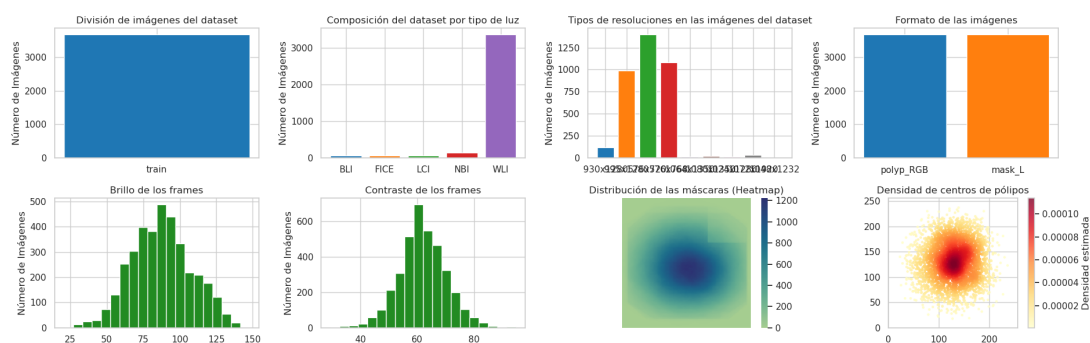


Figura 4.4: Gráficos de la composición del *dataset PolypDB*.

Tabla 4.3: Resumen de la composición de *PolypDB*.

Ítem	Valor
Total de imágenes	3675
Resoluciones	
1280x930	118
1280x995	990
720x576	1399
768x576	1081
1350x1064	17
1350x1080	24
1280x1024	12
1920x1072	33
1048x1232	1
Tipos de luz	
BLI	59
FICE	60
LCI	59
NBI	139
WLI	3358
Volumen medio pólipos / imagen	8.52 %
Volumen medio bbox / imagen	57.07 %
Distancia media centro pólipo / centro imagen	47.32px

Este *dataset* ofrece una variedad de luces y pese a tener los pólipos en una distribución más centrada, el número de datos y la razonable descentralización de los pólipos respecto a la imagen hace que este *dataset* ofrezca exactamente lo que buscamos, un conjunto de datos para enriquecer el *split* de *train*.

4.3. CVC

Este *dataset* público es presentado por primera vez por Jorge Bernal, F. Javier Sánchez y colaboradores en "*WM-DOVA maps for accurate polyp highlighting in colonoscopy: Validation vs. saliency maps from physicians*" [65] junto a otros *datasets* de la familia CVC. Este *dataset* se compone de imágenes obtenidas en centros de Barcelona y Valencia, sumando un total de 612 imágenes. Este *dataset* no incluye una división por tipo de luz ni metadatos clínicos de libre acceso.

Preprocesado de CVC y análisis

Este *dataset* es preprocesado de forma idéntica al *dataset* de PolypDB y Piccolo. De esta forma el *dataset* resultante y su composición se ve en la Figura 4.5 y la Tabla 4.4



Figura 4.5: Gráficos de la composición del *dataset* CVC.

Tabla 4.4: Resumen de la composición de CVC.

Ítem	Valor
Total de imágenes	579
Resoluciones	
384x288	118
Volumen medio pólipo / imagen	9.47 %
Volumen medio bbox / imagen	14.51 %
Distancia media centro pólipo / centro imagen	67.28px

Este conjunto de datos es utilizado al igual que el *dataset* de PolypDB para dar variabilidad al conjunto de *train*.

4.4. Composición de los *splits*

El paso de dividir y distribuir los distintos *datasets* que componen los datos disponibles en *splits* es un paso fundamental. Ya que garantiza una evaluación fiable y generalizable, además de una adecuada independencia de los datos entre *splits* para poder garantizar que los datos con los que se entrenan no tengan relación con los de *validation* y *test* y incurrir en resultados inválidos.

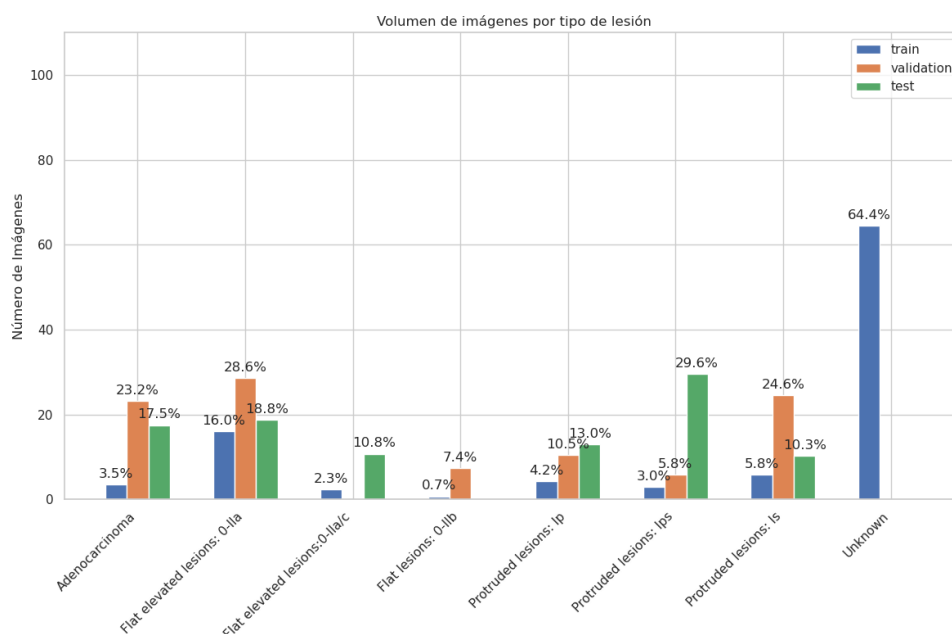


Figura 4.6: Porcentaje de imágenes de cada tipo de lesión en cada *split*.

Para conseguir todo esto hemos fundamentado la división de los datos en el *dataset* de *Piccolo*, teniendo en cuenta este hemos redistribuido las imágenes de tal forma que cada conjunto tenga una representación adecuada de cada tipo de lesión disponible como podemos ver en la Figura 4.6 en la que mostramos que porcentaje de imágenes representa a cada lesión. Puntualizar que el balanceo perfecto no es posible ya que la cantidad de imágenes de cada tipo de lesión es variable y no podemos dividir las imágenes que pertenecen a la misma lesión, otra limitación es el caso de lesiones como la *lesión plana 0-lla* de la cuál solo disponemos de dos lesiones por lo que hemos distribuido una en entrenamiento y otra en *validation*. Por último añadir que además del balanceo teórico, ha habido un balanceo de los pólipos con mejores y peores resultados dentro y fuera del conjunto de *train* para poder incluir en este aquellos pólipos que le resultan más difíciles de aprender.

La composición final de estos *splits* es la que podemos observar en la Figura 4.7 y la Tabla 4.5. De esta forma *Piccolo* es usado como base para garantizar la composición y diversidad de lesiones en *validation* y *test*, mientras que *PolypDB* y *CVC* son utilizadas para dar mayor cantidad de datos a las arquitecturas y así ofrecer una capacidad de generalización eficaz como veremos en las comparaciones de rendimiento del capítulo 5.

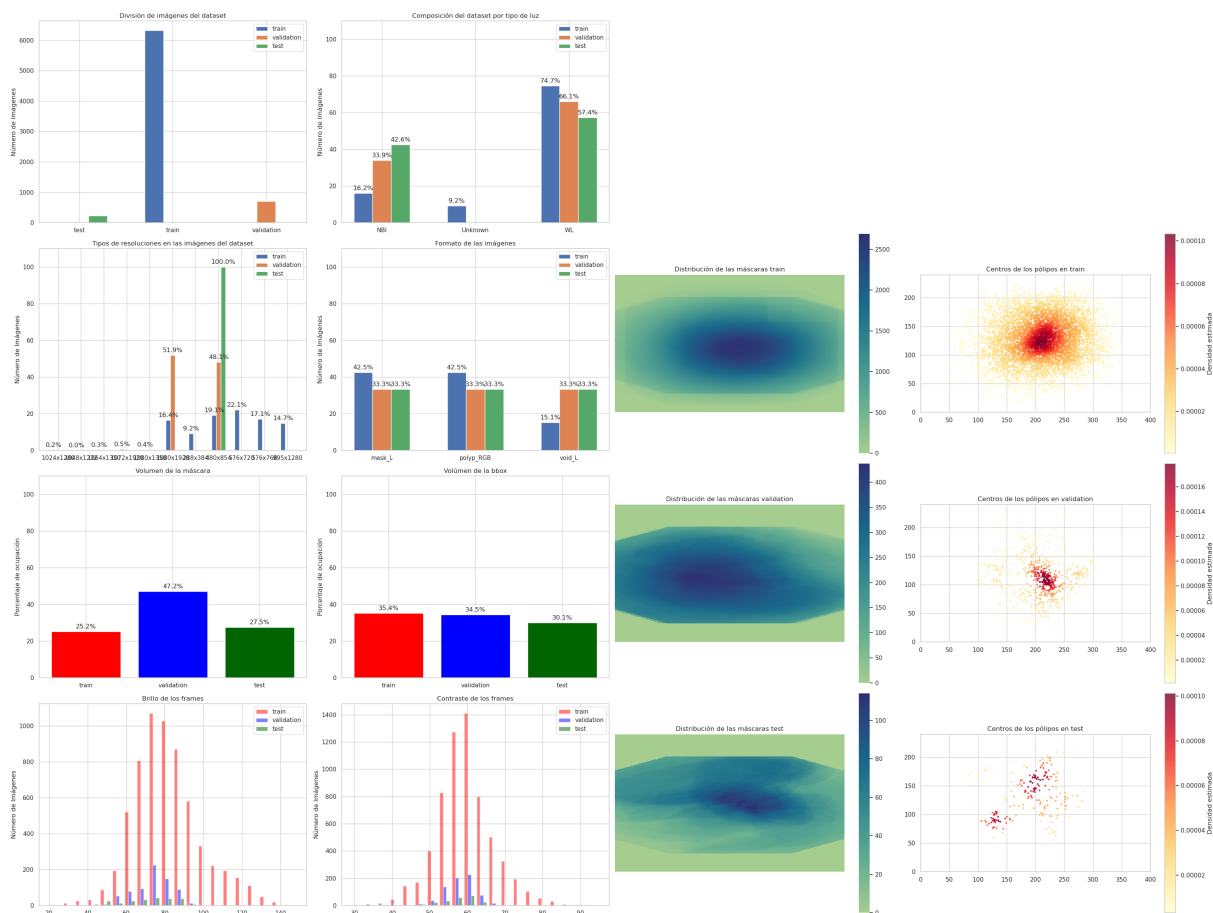


Figura 4.7: Ejemplos de formatos de cajas delimitadoras. (a) Gráficos con información contrastada por *split*; (b) Heatmap con distribución de pólipos y centros por *split*.

Característica	TRAIN	VALIDATION	TEST
Total imágenes	6318	707	223
Resoluciones			
854x480	1208	340	223
1920x1080	1039	367	X
720x576	1398	X	X
768x576	1078	X	X
1280x995	930	X	X
Otras	665	X	X
Tipos de luz			
WL	4717	707	128
NBI	1022	240	95
Sin etiqueta	579	X	X
Tipos de lesión			
Pediculadas Ips	187	41	66
Planas elevadas 0-IIa/c	146	0	24
Planas elevadas 0-IIa	1012	202	42
Sésiles Is	336	174	23
Pediculadas Ip	268	74	29
Adenocarcenoma	223	164	39
Planas 0-IIb	45	52	0
Sin etiqueta	4071	X	X
Volumen medio pólipos	13.13 %	24.56 %	14.33 %
Volumen medio bbox	62.50 %	60.94 %	53.26 %
Distancia media centro (px)	57.82	50.06	56.59

Tabla 4.5: Comparación de *splits* del *dataset*.

Capítulo 5

Experimentos y resultados

En este capítulo se presentan los experimentos realizados a partir de los datos y métodos expuestos a lo largo del TFG. En primer lugar, se describen con detalle los procedimientos específicos aplicados. A continuación, se analizan los resultados obtenidos, comparando el rendimiento de las arquitecturas *transformer* versus las CNN en la detección de pólipos cancerosos en imágenes de colonoscopias, evaluando si estas nuevas arquitecturas representan una evolución significativa frente a las CNN clásicas. Para ello, se utilizan los *benchmarks* definidos previamente y se examina también la eficacia de los datos empleados. Finalmente, se realiza una comparación exhaustiva de los resultados entre todas las arquitecturas estudiadas.

5.1. *Benchmark*

Un *benchmark* es un conjunto de pruebas estandarizadas diseñado para evaluar de forma objetiva y comparable el desempeño de distintos modelos. Estos consisten en una serie de parámetros, datos y métricas predefinidas para obtener resultados comparables.

Los *benchmark* definidos para el análisis de los modelos estudiados en este TFG tienen el objetivo no solo de proporcionar el mismo entorno y métricas para todos los modelos, si no ofrecer además una comparación de la eficiencia de los datos utilizados. Con este fin compararemos el rendimiento de las arquitecturas usando los *benchmarks* definidos a continuación.

5.1.1. *Benchmark Piccolo*

Este es el primer conjunto de datos utilizado y a través de este *benchmark* intentamos proporcionar un rendimiento del mismo en los modelos estudiados. Este *benchmark* se compone de:

- **Dataset:** El *dataset* de Piccolo [1], definido en el capítulo 4.
- **Loss-function:** Se utilizará la función de pérdida "*smooth-l1*" la cuál compara la distancia entre el vector de salida y el objetivo.
- **Optimizador:** En este caso se utiliza *AdamW*, una versión con mayor regularización, adecuada para una *loss-function* más sencilla. Utilizamos *AdamW* con una tasa de aprendizaje experimentalmente adecuada de $5 \cdot 10^{-5}$.
- **Tiempo de entrenamiento:** En este contexto supone durante cuantas épocas se entrena el modelo, esto será generalmente 30 épocas¹ suficientes para obtener el máximo aprendizaje del modelo.

5.1.2. *Benchmark Piccolo+*

Este está compuesto principalmente por el *dataset* de *Piccolo* y respetando la distribución de lesiones del *dataset* original como vemos en la Tabla 4.5. Este *benchmark* trata de mejorar el rendimiento del *benchmark Piccolo* al aumentar la cantidad de datos del conjunto de *train* y ajustando otros parámetros del *benchmark*, dando a los modelos una capacidad mayor de generalización y aprendizaje. Este se compone de:

- **Dataset:** Al tratar de mejorar el rendimiento de Piccolo, este *benchmark* utiliza los *datasets* de *CVC* [14] y *PolypDB* [15] para aportar más datos al conjunto de *train* para intentar arreglar el sobre ajuste, con bastante éxito como podemos ver en los resultados de la Tabla 5.2.

La composición de estos datos se analiza en el capítulo 4.

- **Loss-function:** Con el objetivo de mejorar el rendimiento, he seleccionado como *loss-function* CIoU, siendo esta más adecuada como explico en el capítulo 2. Gracias a esto hemos experimentado **mejoras entre el 5 % y el 10 %**

¹Modelos más simples o antiguos como *ViT-Small* o *YolosBBoxOnly* con 50

- **Optimizador:** Debido a la complejidad de la nueva *loss-fucntion* no es necesario la ligera regularización de *AdamW* y por lo tanto utilizamos *Adam* con una tasa de aprendizaje de $5 \cdot 10^{-5}$.
- **Tiempo de entrenamiento:** Para este *benchmark* y con el fin de ser comparable a los resultados del *benchmark* Piccolo utilizamos el mismo numero de épocas, siendo generalmente 30 épocas² suficientes para obtener el máximo aprendizaje del modelo con las nuevas herramientas.

El *benchmark* Piccolo+CVC+PolypDB se ha diseñado para superar los resultados del conjunto original Piccolo, manteniendo la comparabilidad. Por ello, comparamos ambos *benchmarks* con el fin de evaluar las mejoras introducidas en el conjunto de datos de Piccolo. Finalmente, el *benchmark* Piccolo+CVC+PolypDB es el que utilizamos para analizar las arquitecturas *transformer* en el ámbito de estudio de este TFG.

5.2. Resultados y análisis

En esta sección se presentan los resultados obtenidos durante la realización del TFG, analizando los resultados obtenidos individualmente en cada *benchmark* para posteriormente compararlos y concluir analizando el rendimiento general de las arquitecturas estudiadas. Los resultados expuestos se componen tanto del resultado final del modelo con el conjunto de *test*, como los IoU en la última época tanto de *train* como *validation*, para dar una perspectiva mejor de la capacidad del modelo.

5.2.1. Resultados del *benchmark* Piccolo

Los resultados obtenidos con este primer *benchmark* como hemos ido adelantando, los podemos ver reflejados en la Tabla 5.1 y en la Figura 5.1.

²En modelos más pesados como son *ViT-Large* y *DenseNet* se usan 20 épocas ya que son suficientes para estos modelos y no disponemos de mayor capacidad de cómputo

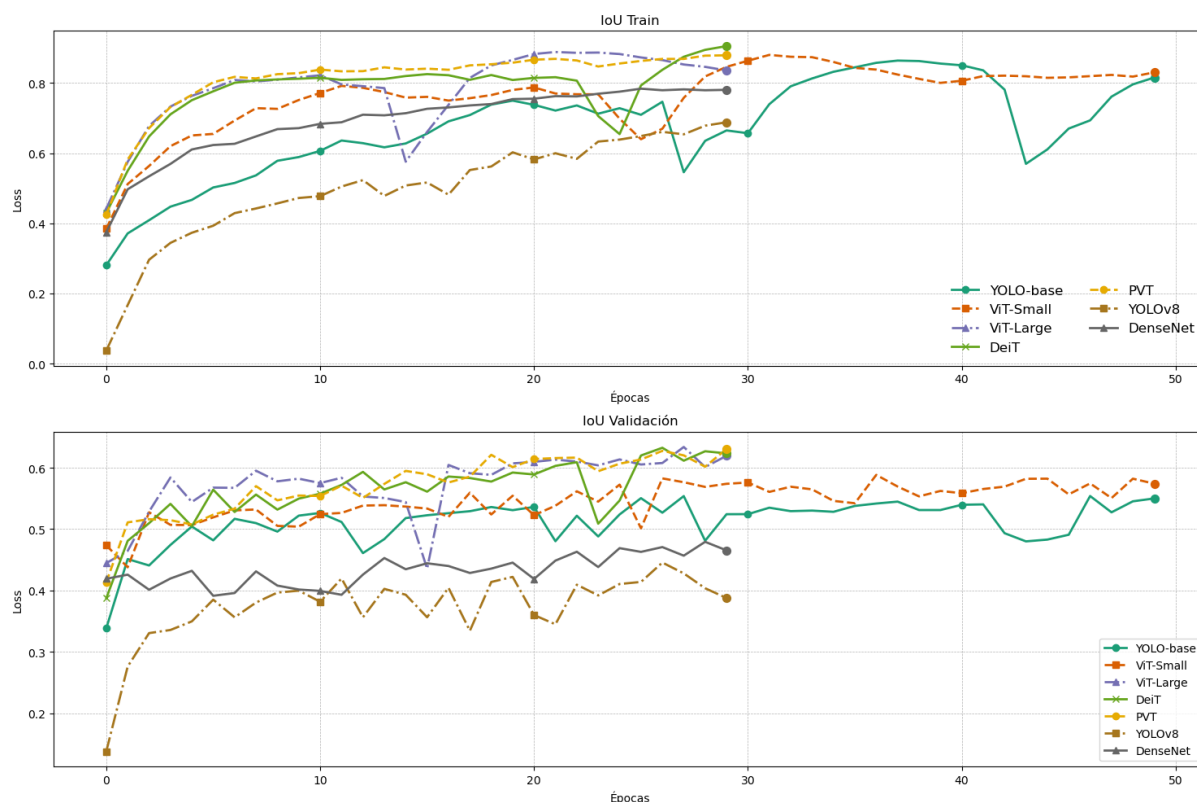


Figura 5.1: Resultados del rendimiento de los modelos en *train* y *validation* en el *benchmark* Piccolo.

Tabla 5.1: Rendimiento de las arquitecturas en el *benchmark* Piccolo.

Modelo	Train (%)	Diferencia $\Leftarrow$	Validation (%)	TEST (%)
YolosBBoxOnly	81.58	26.58	55.00	43.30
ViT-Small	83.06	25.65	57.41	48.25
ViT-Large	83.60	21.66	61.93	66.02
DeiT	90.54	28.21	62.33	65.43
PVT	87.91	24.86	63.05	67.51
Media transformers	84.98	26.34	58.64	54.31
YOLOv8	68.83	30.04	38.79	33.30
DenseNet	78.07	31.53	46.55	45.69
Media CNN	73.45	30.78	42.67	39.49

Podemos observar los resultados de las arquitecturas *transformer* son bastante mejores que el de la arquitectura CNN, destacando el hecho de que las arquitecturas a más modernas mejor resultado, como es el caso de *DeiT* y *PVT*, gracias a sus aproximaciones y arquitecturas

más evolucionadas que sus contrapartes *YolosBBoxOnly* y *ViT-Small*, quedando *ViT-Large* con resultados mejores debido a la complejidad de su red.

Esta tendencia de mejores resultados respecto a la complejidad y modernidad de las arquitecturas es similar en el caso de las CNN, con una sorprendente brecha de 12.39% entre *YOLOv8* y *DenseNet*.

Pese a todo el rendimiento de todos los modelos es generalmente pobre y tiende al overfitting como vemos en la Figura 5.1, con una media de diferencia entre *validation* y *train* en las arquitecturas *transformer* de 26.34%. Aunque en el caso de las CNN ha tenido una mayor dificultad para generalizar los datos con un 30.78% de diferencia entre los *splits train* y *validation*.

Estos resultados tan mediocres seguramente se deban a la poca variabilidad de los datos del *benchmark* Piccolo haciendo estos no del todo adecuados para entrenamiento debido a que tan solo tienen imágenes de 74 lesiones, pero siendo un excelente *dataset* para controlar las muestras de *validation* y *test* adecuadamente, como ya discutimos en el capítulo 4. Estos resultados mejorarán debido a la introducción de nuevos datos en entrenamiento, como veremos más adelante.

5.2.2. Resultados del *benchmark* Piccolo+CVC+PolypDB

Este *benchmark* trata de mejorar los resultados obtenidos en el *benchmark* de Piccolo. Para ello abordamos el *overfitting* visto añadiendo los *datasets* de *CVC* y *PolypDB* dando a los datos una mejor variabilidad en *train* haciendo a los modelos más precisos y consiguiendo mejor generalización, la composición final de estos datos se presenta en el capítulo 4.

Este *benchmark* representa la versión definitiva de los datos y debido a su medida composición de los conjuntos de *validation* y *test*, los resultados de este *benchmark* representan el rendimiento definitivo de los modelos estudiados para el problema abordado en este TFG. Los resultados obtenidos con este nuevo conjunto de datos podemos verlos en la Tabla 5.2 y la Figura 5.2.

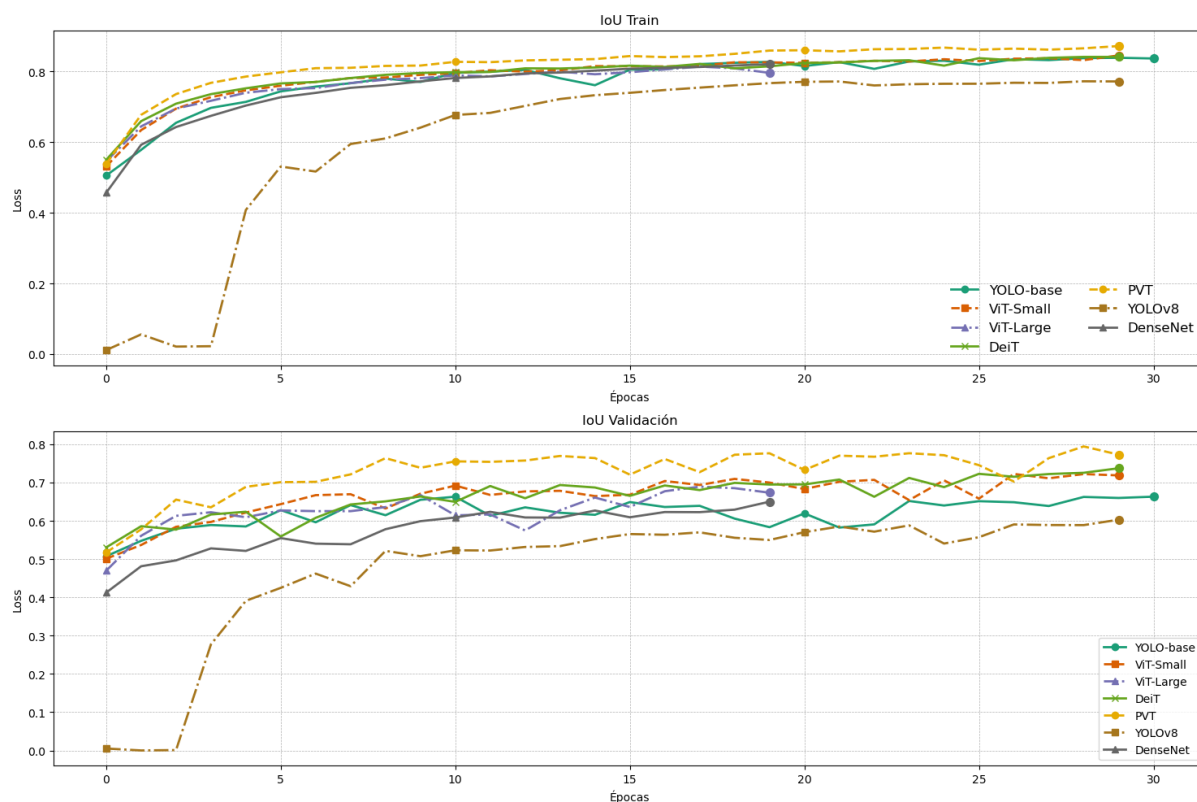


Figura 5.2: Resultados del rendimiento de los modelos en *train* y *validation* en el *benchmark* Piccolo+CVC+PolypDB.

Tabla 5.2: Rendimiento de las arquitecturas en el *benchmark* Piccolo+CVC+PolypDB.

Modelo	Train (%)	Diferencia $\Leftrightarrow$	Validation (%)	TEST (%)
YolosBBoxOnly	83.72	17.41	66.31	75.38
ViT-Small	84.59	12.72	71.87	78.02
ViT-Large	79.64	12.30	67.34	75.13
DeiT	84.17	10.44	73.73	81.10
PVT	87.24	9.95	77.29	82.26
Media transformers	83.87	12.56	71.31	78.38
YOLOv8	77.22	16.93	60.29	65.25
DenseNet	82.11	17.11	65.00	67.18
Media CNN	79.66	17.02	62.64	66.22

Los resultados obtenidos superan ampliamente al *benchmark* Piccolo. En concreto podemos observar como las arquitecturas *transformer* presentan una mejora promedio del 44.32 % mientras que las CNN mejoran en un 26.73 % respecto al *benchmark* anterior. Estas mejoras

evidencian que una mayor diversidad en los datos es clave para lograr un entrenamiento eficaz, donde las arquitecturas puedan generalizar correctamente y alcanzar una precisión adecuada. La fiabilidad de estos resultados está garantizada por los conjuntos de *validation* y *test* debidamente seleccionados en el capítulo 4.

El rendimiento de las arquitecturas *transformer* varía considerablemente en función de su diseño. Se puede observar una tendencia clara, cuanto más moderna y rica en nuevas tecnologías es la arquitectura, mejores son sus resultados. Si comparamos modelos como *YolosBBoxOnly*, *ViT-Large* y *ViT-Small*, cuyos resultados rondan el 75 %, con las arquitecturas más modernas *PVT* y *DeiT*, con un 82.26 % y 81.1 % respectivamente, lo que representa una mejora aproximada del 5.5 %. Esta tendencia también se aprecia en las arquitecturas CNN, ya que *DenseNet* tiene un resultado un 1.93 % mejor que *Yolov8*. También dando un *overfitting* mayor.

En conclusión, si observamos todos los datos podemos afirmar que, si bien las primeras versiones de ViT ofrecen resultados en torno al 10 % mejores a las CNN y con un *overfitting* en los casos de arquitecturas más antiguas similar (17 %), los *Transformers* más avanzados, como *DeiT* y *PVT*, destacan notablemente con resultados al menos un 14 % mejores que las CNN. Esto confirma que las arquitecturas ViT modernas no solo superan a las CNN como podemos ver en la Figura 5.2, sino que también tienen el potencial de mejorar significativamente las herramientas clínicas actuales.

Capítulo 6

Conclusiones y líneas futuras

6.1. Conclusiones

A lo largo de este TFG se ha demostrado el potencial de las arquitecturas basadas en *Transformers*, especialmente los modelos *ViT* modernos, para la detección automática de pólipos en imágenes de colonoscopia. Se ha realizado una comparación exhaustiva del rendimiento de estos modelos, comparando su rendimiento con distintos conjuntos de datos frente a arquitecturas CNN, más habituales en la industria, para esta comparación se han seleccionado CNN populares como *YOLOv8* o *DenseNet*, esta última reconocida como la mejor en imagen médica en la publicación *Comparison of Vision Transformers and Convolutional Neural Networks in Medical Image Analysis: A Systematic Review* [11].

Esto evidencia que, si bien los modelos *ViT* clásicos ofrecen mejoras limitadas, las arquitecturas *ViT* más recientes como *DeiT* y *PVT* ofrecen rendimientos significativamente superiores a las CNN en la tarea de detección que aborda este TFG.

La incorporación de un *benchmark* más robusto y diverso, combinando bases de datos como *Piccolo* [1], *CVC* [14] y *PolypDB* [15], ha permitido mejorar las capacidades de generalización de los modelos evaluados. Este aumento en la variabilidad de datos ha resultado clave, especialmente para las arquitecturas tipo *Transformer*, debido a su arquitectura y elementos como las cabezas de atención que precisan de grandes cantidades de datos para su correcto entrenamiento.

Los resultados muestran que, con una mayor diversidad de datos, los *Transformers* pueden mejorar hasta en un 44.32 % el rendimiento respecto al *benchmark* *Piccolo* original, Además estas superan en un 12.16 % de media a los resultados de las CNN y presentan una adaptación mejor a los datos con un 4.46 % menos de *overfitting* que las CNN. Estos datos confirman que

las arquitecturas ViT modernas son capaces de superar a las CNN tradicionales en la tarea de detección de pólipos, evidenciando el potencial que tiene la incorporación de estas arquitecturas a los programas de cribado de CCR, mejorando la detección precoz y, en consecuencia, el pronóstico de los pacientes.

No obstante, este estudio presenta algunas limitaciones que deben ser consideradas. En primer lugar, las arquitecturas ViT, aunque más precisas, requieren mayores cantidades de datos y por lo tanto recursos computacionales y tiempo de entrenamiento. Además, algunos modelos más antiguos presentan problemas de *overfitting*. Estos aspectos limitan su aplicación y abren líneas interesantes para futuros desarrollos.

6.2. Líneas futuras

Este trabajo, debido a su naturaleza de TFG, representa un acercamiento inicial al análisis comparativo de arquitecturas ViT y CNN para la detección de pólipos. A partir del trabajo realizado y los resultados obtenidos, se proponen las siguientes líneas de desarrollo futuro:

- **Detección múltiple de pólipos:** Adaptar los modelos para la detección simultánea de múltiples lesiones por imagen, esta funcionalidad es común en arquitecturas como YOLO. Esta mejora permitiría representar de forma más realista escenarios clínicos, donde se pueden encontrar múltiples lesiones en la misma imagen.
- **Clasificación binaria previa a la detección:** Este TFG se limita a la tarea de detección, una mejora sustancial sería la implementación de cabezas de clasificación para determinar la presencia o ausencia de pólipos. Esta modificación es esencial para la aplicación de las arquitecturas en un entorno clínico. Permitiría en trabajos que traten de crear arquitecturas propias con este objetivo, podría tratarse de una etapa anterior a la detección, para determinar primero si hay un pólipo antes de aplicar modelos más complejos de detección, optimizando el uso de recursos computacionales. En este caso se deberían adecuar además una función de pérdida adecuada a la tarea de detección condicionada por la de clasificación.
- **Evaluación de nuevas arquitecturas:** La base de software desarrollada durante este trabajo permite extender fácilmente el análisis a modelos adicionales como *Swin Transformer*, *SegFormer* o *ConvNext*, así como arquitecturas híbridas que combinan CNNs y *Transformers*. Estas versiones no entraban dentro del marco de este TFG, pero su estudio

puede ser desarrollado con las mismas herramientas y utilizando como referencia este trabajo presente en el GitHub de este TFG [49].

- **Ampliación del *dataset* y enriquecimiento de condiciones clínicas:** Incorporar imágenes de mayor resolución, variaciones anatómicas, condiciones de iluminación extremas y casos clínicos menos frecuentes permitiría aumentar la robustez del modelo y su capacidad de generalización.
- **Aplicación en tiempo real y validación clínica:** Una línea crucial es la adaptación de los modelos para su ejecución en tiempo real y su validación en entornos clínicos reales, con la colaboración de profesionales médicos. Esto incluiría pruebas de campo, análisis del flujo de trabajo y estudios clínicos controlados.
- **Aplicación en tiempo real y validación clínica:** Una línea crucial es la adaptación de los modelos evaluados para su integración en entornos clínicos reales, con la colaboración de profesionales médicos. Esto incluiría pruebas de campo, análisis del flujo de trabajo y estudios clínicos controlados.

Este TFG constituye una pequeña aportación al progreso de la visión artificial e IA en el ámbito médico. Espero que este TFG pueda resultar útil a cualquier investigación presente o futura que ayude a cumplir el objetivo final de todo el esfuerzo hecho en este campo. El desarrollo y aplicación de nuevas tecnologías, que generen un impacto real en un sector tan importante como la salud en general y la oncología en particular.

Bibliografía

- [1] Biobanco Vasco, “Piccolo rgb/nbi (widefield) image collection,” 2025. [Online]. Available: <https://www.biobancovasco.bioef.eus/en/Sample-and-data-e-catalog/Databases/PD178-PICCOLO-EN1.html>
- [2] M. Gharibi, “The magic behind embedding models,” 2019. [Online]. Available: <https://medium.com/data-science/the-magic-behind-embedding-models-c3af62f71fb>
- [3] World Health Organization, “Colorectal cancer – fact sheet,” 2024. [Online]. Available: <https://www.who.int/news-room/fact-sheets/detail/colorectal-cancer>
- [4] S. E. nola de Oncología Médica (SEOM), “Las cifras del cáncer en españa 2024,” 2024. [Online]. Available: https://www.geicam.org/wp-content/uploads/2024/03/LAS_CIFRAS_2024.pdf
- [5] X. Zhang, H. Li, and H. Lu, “Colorectal image analysis for polyp diagnosis via pam-net,” *Frontiers in Computational Neuroscience*, 2024.
- [6] P. Solís-Muñoz, J. A. Solís-Herruzo, and S. Rodríguez-Muñoz, “Experience of the endoscopist increases detection rates of smaller size and higher histological grade polyps,” *Journal of Gastroenterology and Hepatology*, 2014.
- [7] D. K. Jung, T. O. Kim, M. S. Kang, M. S. Kim, M. S. Kim, and Y. S. Moon, “The colonoscopist’s expertise affects the characteristics of detected polyps,” *Clinical Endoscopy*, 2016.
- [8] K. ElKarazle, V. Raman, P. Then, and C. Chua, “Detection of colorectal polyps from colonoscopy using machine learning: A survey on modern techniques,” 2023.
- [9] M. Yeung, E. Sala, C.-B. Schönlieb, and L. Rundo, “Focus u-net: A novel dual attention-gated cnn for polyp segmentation during colonoscopy,” 2021.

- [10] J. e. a. Yoon, “A prospective multicenter randomized controlled trial on artificial intelligence–assisted colon polyp detection using retinanet,” *Scientific Reports*, vol. 14, p. 77079, 2024.
- [11] S. Takahashi, Y. Sakaguchi, N. Kouno, H. Yamamoto, A. Sato, and K. Tanaka, “Comparison of vision transformers and convolutional neural networks in medical image analysis: A systematic review,” *Journal of Medical Systems*, vol. 48, no. 84, 2024.
- [12] B. Dong, W. Wang, D.-P. Fan, J. Li, H. Fu, and L. Shao, “Polyp-pvt: Polyp segmentation with pyramid vision transformers,” *arXiv preprint arXiv:2108.06932*, 2021. [Online]. Available: <https://arxiv.org/pdf/2108.06932>
- [13] L. F. Sánchez-Peralta, J. B. Pagador, A. P. Ruiz, Ángel José Calderón, F. Polo, N. Andraka, R. Bilbao, B. Glover, C. L. Saratxaga, and F. M. Sánchez-Margallo, “Piccolo white-light and narrow-band imaging colonoscopic dataset: A performance comparative of models and datasets,” *Applied Sciences*, vol. 10, no. 23, p. 8501, 2020. [Online]. Available: https://www.mdpi.com/2076-3417/10/23/8501?utm_source=chatgpt.com
- [14] D. Jha, P. H. Smedsrud, M. A. Riegler, D. Johansen, T. de Lange, P. Halvorsen, and H. D. Johansen, “Resunet++: An advanced architecture for medical image segmentation,” *Proceedings of the 2019 IEEE International Symposium on Multimedia (ISM)*, 2019.
- [15] D. Jha, T. de Lange, U. Bagci, M. Riegler, and P. Halvorsen, “Polypdb: A multi-center and multi-modality dataset for polyp detection and segmentation,” <https://osf.io/pr7ms/>, 2025.
- [16] American Cancer Society, “¿qué es el cáncer de colon o de recto?” <https://www.cancer.org/es/cancer/tipos/cancer-de-colon-o-recto/acerca/que-es-cancer-de-colon-o-recto.html>, 2024.
- [17] “Definición de colon,” 2025, instituto Nacional del Cáncer (NCI). [Online]. Available: <https://www.cancer.gov/espanol/publicaciones/diccionarios/diccionario-cancer/def/colon>
- [18] Regional Cancer Care Associates, “Signos precancerosos de cáncer de colon a tener en cuenta.” [Online]. Available: <https://regionalcancercare.org/es/news/signs-of-precancerous-colon-cancer/>
- [19] M. J. O’Brien and D. Gibbons, “The adenoma-carcinoma sequence in colorectal neoplasia,” *Surgical Oncology Clinics of North America*, vol. 5, no. 3, 1996. [Online]. Available: <https://pubmed.ncbi.nlm.nih.gov/8829316/>

- [20] N. Autor and N. OtroAutor, "Different types of colonic polyps with hematoxylin-eosin staining," *International Journal of Colorectal Disease*, imagen adaptada de ResearchGate. [Online]. Available: https://www.researchgate.net/figure/Different-types-of-colonic-polyps-with-hematoxylin-eosin-staining_fig1_322082763
- [21] Participants in the Paris Workshop, "The paris endoscopic classification of superficial neoplastic lesions: esophagus, stomach, and colon," *Gastrointestinal Endoscopy*, vol. 58, no. 6 Suppl, pp. S3–S43, Dec. 2003, clasificación París para lesiones neoplásicas superficiales.
- [22] Sociedad Aragonesa de Patología Digestiva (SAPD), "Caracterización de los pólipos en el colon en la práctica clínica y su implicación terapéutica." [Online]. Available: <https://www.sapd.es/documents/uploads/Presentacion-Polipos1.pdf>
- [23] M. S. Hossain, H. Karuniawati, A. A. Jairoun, Z. Urbi, J. Ooi, A. John, F. Al Dhaheri, Y. Al Dhaheri, E. Abu-Gharbieh, and N. Sultana, "Colorectal cancer: A review of carcinogenesis, global epidemiology, current challenges, risk factors, preventive and treatment strategies," *Cancers*, vol. 14, no. 7, p. 1732, 2022. [Online]. Available: <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC8996939/>
- [24] H.-H. C. Chiu, G. H.-H. Jen, Y.-W. Wang, J.-C. Fann, C.-Y. Hsu, Y.-C. Jeng, A. M.-F. Yen, S. Y.-H. Chiu, S.-L. Chen, W.-F. Hsu, Y.-C. Lee, M.-S. Wu, C.-Y. Wu, Y.-Y. Jou, and T. H.-H. Chen, "Long-term effectiveness of faecal immunochemical test screening for proximal and distal colorectal cancers," *Gut*, vol. 70, no. 12, 2021.
- [25] A. Castells, E. Quintero, F. J. Rodríguez-Bigas, J. M. Bujanda, J. M. Sánchez, J. Sanz, J. M. García, J. M. García, J. M. García, and J. M. García, "Colorectal cancer screening: Fit for purpose?" *The Lancet*, vol. 405, 2025. [Online]. Available: [https://www.thelancet.com/journals/lancet/article/PIIS0140-6736\(25\)00619-1/fulltext](https://www.thelancet.com/journals/lancet/article/PIIS0140-6736(25)00619-1/fulltext)
- [26] National Cancer Institute, "Fecal immunochemical test," 2025, accedido: 2025-06-02. [Online]. Available: <https://www.cancer.gov/publications/dictionaries/cancer-terms/def/fecal-immunochemical-test>
- [27] J. Makar, J. Abdelmalak, D. Con, B. Hafeez, and M. Garg, "Use of artificial intelligence improves colonoscopy performance in adenoma detection: a systematic review and meta-analysis," *Gastrointestinal Endoscopy*, pp. 1–12, 2024.
- [28] S. Raschka, Y. H. Liu, and V. Mirjalili, *Machine Learning with PyTorch and Scikit-Learn*. Birmingham, UK: Packt Publishing, 2022. [Online]. Available: <https://sebastianraschka.com/books/machine-learning-with-pytorch-and-scikit-learn/>

- [29] W. S. McCulloch and W. Pitts, “A logical calculus of the ideas immanent in nervous activity,” *The Bulletin of Mathematical Biophysics*, vol. 5, no. 4, pp. 115–133, 1943.
- [30] F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, 1958.
- [31] B. Widrow, “Adaline and madaline - 1963,” in *Proceedings of the IEEE First International Conference on Neural Networks*, vol. 1. San Diego, CA: IEEE, 1987, pp. 145–157.
- [32] IBM, “¿qué son las redes neuronales?” 2023. [Online]. Available: <https://www.ibm.com/mx-es/think/topics/neural-networks>
- [33] A. Navlani, “Multi-layer perceptron neural network using python,” 2021. [Online]. Available: <https://machinelearninggeek.com/multi-layer-perceptron-neural-network-using-python/>
- [34] R. Bommasani, D. A. Hudson, E. Adeli, and Altman, “On the opportunities and risks of foundation models,” *arXiv preprint arXiv:2108.07258*, 2021. [Online]. Available: <https://arxiv.org/abs/2108.07258>
- [35] D. Bergmann, “What is fine-tuning?” 2024. [Online]. Available: <https://www.ibm.com/think/topics/fine-tuning>
- [36] IBM, “¿qué es la segmentación de imágenes?” 2025. [Online]. Available: <https://www.ibm.com/es-es/topics/image-segmentation>
- [37] —, “¿qué es la detección de objetos?” 2025. [Online]. Available: <https://www.ibm.com/mx-es/think/topics/object-detection?>
- [38] Google Developers, “Dividing the original dataset,” 2025. [Online]. Available: <https://developers.google.com/machine-learning/crash-course/overfitting/dividing-datasets>
- [39] M. Hollemans, “Beyond classification,” 2025. [Online]. Available: <https://www.kodeco.com/books/machine-learning-by-tutorials/v2.0/chapters/9-beyond-classification>
- [40] Wikipedia contributors, “Error absoluto medio – wikipedia, la enciclopedia libre,” 2025. [Online]. Available: https://es.wikipedia.org/wiki/Error_absoluto_medio
- [41] P. Chandra, “Iou loss functions for faster & more accurate object detection,” 2023. [Online]. Available: <https://learnopencv.com/iou-loss-functions-object-detection/>

- [42] B. Widrow and M. E. Hoff, “Adaptive switching circuits,” in *1960 IRE WESCON Convention Record, Part 4*. New York: Institute of Radio Engineers, 1960, pp. 96–104.
- [43] R. Zhao, D. Morwani, D. Brandfonbrener, N. Vyas, and S. Kakade, “Deconstructing what makes a good optimizer for language models,” *arXiv preprint arXiv:2407.07972*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.07972>
- [44] T. Mucci, “Overfitting vs. underfitting: Finding the balance,” 2024. [Online]. Available: <https://www.ibm.com/think/topics/overfitting-vs-underfitting>
- [45] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Kaiser, and I. Polosukhin, “Attention is all you need,” vol. 30, 2017.
- [46] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations (ICLR)*, 2021. [Online]. Available: <https://openreview.net/forum?id=YicbFdNTTy>
- [47] S. Takahashi, Y. Sakaguchi, N. Kouno, K. Takasawa, K. Ishizu, Y. Akagi, R. Aoyama, N. Teraya, A. Bolatkan, N. Shinkai, H. Machino, K. Kobayashi, K. Asada, M. Komatsu, S. Kaneko, M. Sugiyama, and R. Hamamoto, “Comparison of vision transformers and convolutional neural networks in medical image analysis: A systematic review,” *Journal of Medical Systems*, vol. 48, 2024.
- [48] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimeshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” *arXiv preprint arXiv:1912.01703*, 2019. [Online]. Available: <https://arxiv.org/abs/1912.01703>
- [49] S. P. Núñez, “Detección de pólipos aplicando vision transformers,” 2025. [Online]. Available: https://github.com/sprieton/deteccion_de_polipos_aplicando_ViT
- [50] NVIDIA Corporation, “Nvidia a40 data center gpu for visual computing,” 2025. [Online]. Available: <https://www.nvidia.com/es-es/data-center/a40/>
- [51] PyTorch Contributors, “torch.utils.data — pytorch documentation,” 2025. [Online]. Available: <https://pytorch.org/docs/stable/data.html>

- [52] Torchvision Contributors, “torchvision.ops.complete_box_iou_loss,” 2025. [Online]. Available: https://docs.pytorch.org/vision/main/generated/torchvision.ops.complete_box_iou_loss.html
- [53] PyTorch Contributors, “torch.optim.adam,” 2025. [Online]. Available: <https://pytorch.org/docs/stable/generated/torch.optim.Adam.html>
- [54] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, “You only look once: Unified, real-time object detection,” *arXiv preprint arXiv:1506.02640*, 2016. [Online]. Available: <https://arxiv.org/abs/1506.02640>
- [55] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, “Microsoft coco: Common objects in context.” [Online]. Available: <https://cocodataset.org/#home>
- [56] Y. Fang, B. Liao, X. Wang, J. Fang, J. Qi, R. Wu, J. Niu, and W. Liu, “You only look at one sequence: Rethinking transformer in vision through object detection,” *arXiv preprint arXiv:2106.00666*, 2021. [Online]. Available: <https://arxiv.org/abs/2106.00666>
- [57] H. V. Lab, “YOLOS-Base: You only look at one sequence - base model,” <https://huggingface.co/hustvl/yolos-base>, 2021, modelo fine-tuned en COCO 2017 para detección de objetos.
- [58] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics yolov8,” 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [59] G. Boesch, “Yolov8: A complete guide,” December 2024. [Online]. Available: <https://viso.ai/deep-learning/yolov8-guide/>
- [60] P. I. M. (TIMM), “Vision transformer (vit),” 2025. [Online]. Available: <https://pprp.github.io/timm/models/vision-transformer/>
- [61] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” *arXiv preprint arXiv:1608.06993*, 2016. [Online]. Available: <https://arxiv.org/pdf/1608.06993>
- [62] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training data-efficient image transformers & distillation through attention,” 2021.
- [63] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao, “Pyramid vision transformer: A versatile backbone for dense prediction without convolutions,”

- in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [64] D. Jha, N. K. Tomar, V. Sharma, Q. Trinh, K. Biswas, H. Pan, R. K. Jha, G. Durak, A. Hann, J. Varkey, H. V. Dao, L. V. Dao, B. P. Nguyen, N. Papachrysos, B. Rieders, P. Thelin Schmidt, E. Geissler, T. Berzin, P. Halvorsen, M. A. Riegler, T. de Lange, and U. Bagci, “Polypdb: A curated multi-center dataset for development of ai algorithms in colonoscopy,” *arXiv preprint arXiv:2409.00045*, 2024. [Online]. Available: <https://arxiv.org/abs/2409.00045>
- [65] J. Bernal, F. J. Sánchez, G. Fernández-Esparrach, D. Gil, C. Rodríguez, and F. Vilariño, “WM-DOVA maps for accurate polyp highlighting in colonoscopy: Validation vs. saliency maps from physicians,” *Computerized Medical Imaging and Graphics*, vol. 43, pp. 99–111, 2015.